

이미지 텐서와 데이터 공급

CNN · 챕터 1 / 16

2일 흐름

언제	무엇
1일 오전	이미지를 숫자로 읽고, 가장 단순한 모델 보기
1일 오후	그 모델을 직접 부수기 → 합성곱
2일 오전	CNN 완성 · 학습과 평가
2일 오후	LeNet · AlexNet · VGG · ResNet

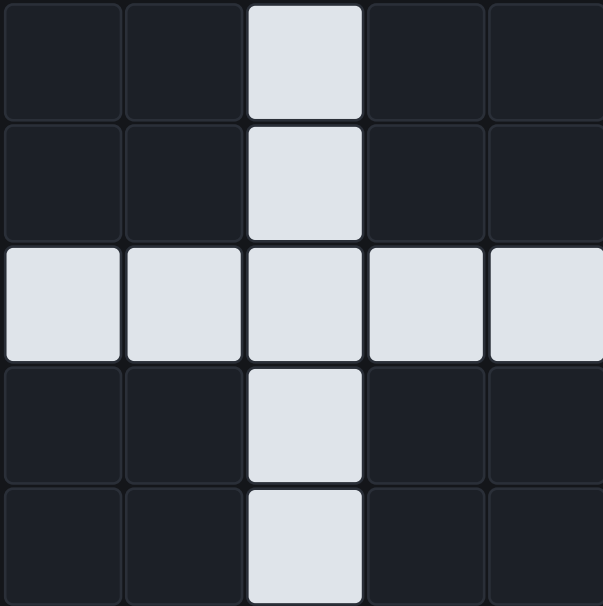
오늘 만드는 모델은
오후에 부술 대상임

오늘의 질문

이미지 한 장은

컴퓨터에게 어떤 모양인가?

이미지는 그림이 아니라 숫자가 놓인 자리표



사람이 보는 것 — 십자

같은 것 →

0	0	1	0	0
0	0	1	0	0
1	1	1	1	1
0	0	1	0	0
0	0	1	0	0

컴퓨터가 받는 것 — 값과 자리

· 값 = 밝기 · 자리 = 위치 · 둘 다 있어야 의미가 됨

이미지를 만드는 두 가지

- 값 — 밝기
- 자리 — 위치

같은 개수의 값이라도 자리가 다르면 다른 이미지임

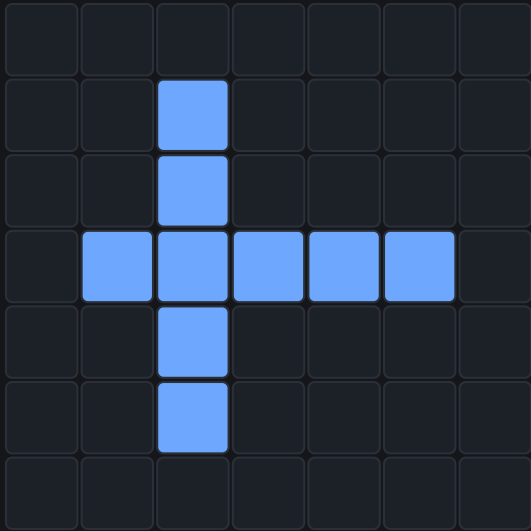
“ 값의 범위는 전처리 선택임. 이미지의 고정 속성 아님 ”

같은 십자를 한 칸 옮기면

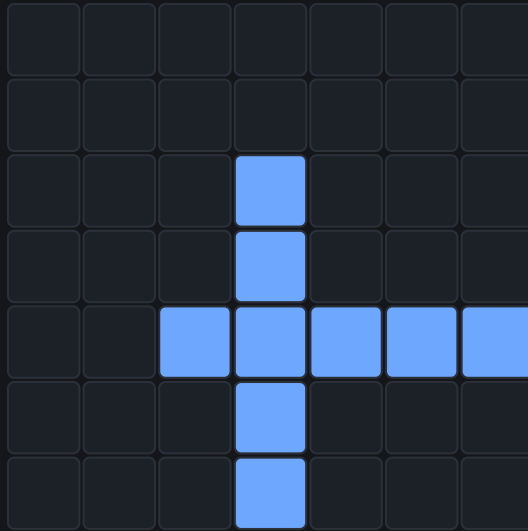
모델은 이 둘을

같이 볼까, 다르게 볼까?

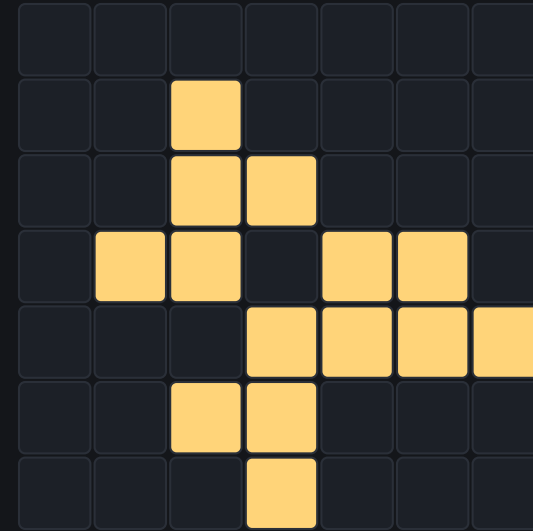
같은 모양인데, 한 칸 옮기면?



원래 십자 · 10이 9개



한 칸 이동 · 10이 9개



값이 다른 자리 · 14개

사람에게는 같은 십자. 숫자 표로는 49자리 중 14자리가 어긋남.
모델은 이 둘을 같게 볼까, 다르게 볼까?

오늘 답하지 않음

사람에게는 같은 십자
숫자 표로는 49자리 중 14자리가 어긋남

오후에 부술 모델이 바로 이 지점에서 문제가 생김

이미지 한 장의 숫자 개수



실무에서 흔한 224×224 컬러 한 장 = 15만 개가 넘는 숫자

다음 질문

shape에 숫자가 네 개면

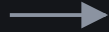
각각 무엇인가?

축은 하나씩 늘어난다

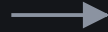
이미 아는 것에서 출발한다



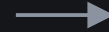
정형 데이터
(행, 열)
축 2개



흑백 이미지
(H, W)
축 2개



RGB 이미지
(C, H, W)
축 3개



여러 장
(N, C, H, W)
축 4개

N = 몇 장 C = 한 자리에 값이 몇 개 H·W = 어디에

PyTorch 이미지 기본 형태

(N, C, H, W)

N	몇 장
C	한 자리에 값이 몇 개
H · W	어디에

축을 하나씩 벗겨내기

x	(8, 1, 28, 28)	전체 배치
x[0]	(1, 28, 28)	첫 장
x[0,0]	(28, 28)	격자
x[0,0,5]	(28,)	한 줄
x[0,0,5,10]	()	값 하나
x[:, :, 10:14, 10:14]	(8, 1, 4, 4)	가운데 4x4

번호 하나를 집으면 그 축은 사라짐
범위로 자르면 축은 남음

자주 하는 실수

(3, 28, 28) 을 **28×28 이미지 3장**으로 읽는 것

- 축이 3개면 앞자리는 장수가 아니라 **채널**
- 이걸 컬러 이미지 **한 장**
- 3장이면 (3, 3, 28, 28)

“ 축 개수를 먼저 세고, 그다음 각 자리를 읽기

”

축을 잘못 세면

(28, 28) 을 Conv2d에 넣으면

```
RuntimeError: Expected 3D (unbatched) or 4D (batched)
input to conv2d, but got input of size: [28, 28]
```

기대한 것과 실제 받은 것을 그대로 말해 줌

다음 질문

채널이 1일 때와 3일 때
무엇이 달라지는가?

채널은 장수가 아니라 층수



같은 자리 위에 값이 쌓여 있음

흑백 (1, 28, 28) → 784개

컬러 (3, 28, 28) → 2352개

자리 수는 같고, 층만 3겹

한 자리의 색을 알려면

세 층을 다 봐야 함

빨강 층만 봐서는 무슨 색인지 모름

채널은 장수가 아니라

총수임

흑백을 3채널로 맞출 때

같은 층을 세 번 복사함

$(1, 28, 28) \rightarrow (3, 28, 28)$
784개 2352개

숫자만 3배. 정보는 그대로임

“ 크기가 커졌다고 아는 게 늘어난 것은 아님 ”

남는 질문

층이 3개면

계산도 층마다 따로 하나?

2일차에 이 질문으로 한 시간을 씀

다음 질문

진짜 데이터도

예상한 모양인가?

FashionMNIST

옷·신발 흑백 사진 · 10가지

- 한 장 (1, 28, 28) · float32
- 정답은 값이 아니라 번호 → 정수
- 흑백이라 오늘 배운 것을 그대로 확인 가능

“ 이 크기는 이 데이터셋의 사실임. 모든 이미지가 이렇다는 뜻 아님

”

데이터셋이 약속하는 것

1. 전체가 **몇 개**인지 말할 수 있음
2. **번호를 주면** 그 자리 데이터를 하나 내놓음

이 둘뿐임

이 단순한 약속 덕분에 배치 나누기가 가능해짐

변환은 언제 일어나나

만든 직후 변환 횟수:	0
0번 한 번 꺼낸 뒤:	1
1,2번 더 꺼낸 뒤:	3

만들 때가 아니라 **꺼낼 때마다 그 한 장만** 변환됨

6만 장을 미리 다 변환해 들고 있으면 감당 안 됨

그런데

원본은 사진 파일이었는데

어떻게 텐서가 됐나?

ToTensor — 이름은 하나, 하는 일은 셋



모델에 넣을 때는 (C, H, W) · 눈으로 볼 때만 (H, W, C)로 되돌림

255로 나누면

원래 값	[0, 64, 128, 255]
나눈 값	[0.0, 0.251, 0.502, 1.0]

순서와 비율은 그대로. **눈금만 바뀜**

정수 → 실수로 바뀌면 메모리는 **4배**

uint8	150,528	바이트
float32	602,112	바이트

다음 질문

60,000장을

어떻게 공급하는가?

손으로 세어 보기

$$60000 \div 64 = ?$$

$$60000 = 64 \times \text{----} + \text{----}$$

배치는 몇 개?

60,000장을 어떻게 공급하는가



$$60000 = 64 \times 937 + 32$$

drop_last=False → 배치 938개, 마지막은 32장

모든 배치 크기가 같다고 가정한 코드는 마지막 배치에서 깨짐

자주 터지는 오류

모든 배치 크기가 같다고 가정한 코드는 마지막 배치에서 깨짐

```
images.shape[0]          # 이렇게 읽기
```

배치 크기를 숫자로 적어두지 않기

오늘 정리

이미지	값 + 자리
축	(N, C, H, W)
채널	장수 아님, 총수
ToTensor	축 순서 · 자료형 · 값 범위
배치	나눗셈. 나머지는 남음

지금 가진 것과 필요한 것

(64, 1, 28, 28) → ? → (64, 10)
지금 손에 있음 비어 있음 필요한 것

784개를 일렬로 펴서

그대로 넣으면 되지 않을까?

일렬로 편 픽셀로 만드는 판별 모델

CNN · 챕터 2

앞 시간에 남긴 자리

(64, 1, 28, 28) → ? → (64, 10)
손에 있음 비어 있음 필요한 것

이 가운데를 오늘 채움

오늘의 질문

픽셀을 일렬로 펴서

그대로 분류하면 되지 않나?

오늘 지나갈 네 가지

펴기	격자를 한 줄로
곱하기	한 줄에 가중치를 걸기
꺼기	층이 층으로 남게 하기
세기	이 모델이 얼마나 큰지

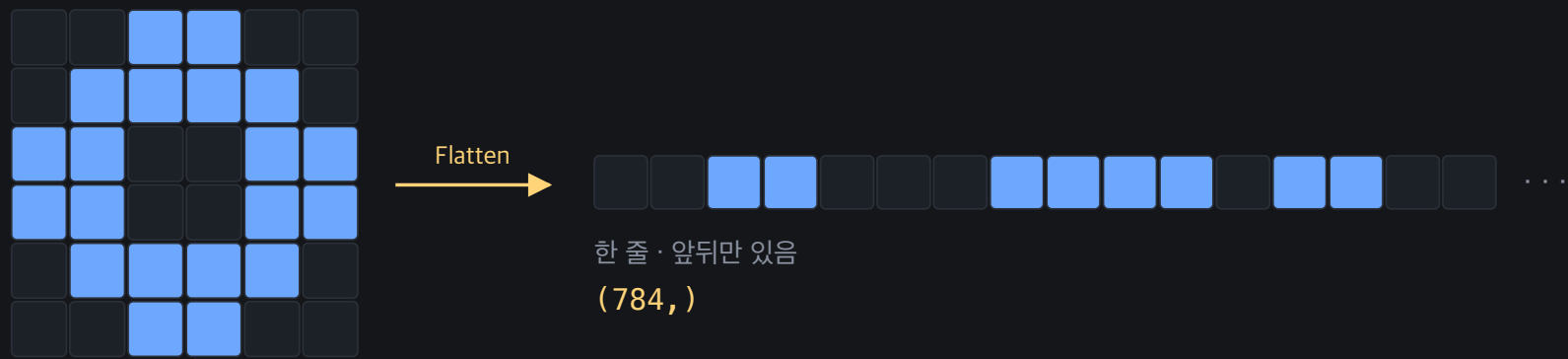
첫 번째 질문

계산하는 층은

어떤 모양을 받는가?

퍼기 — 격자를 한 줄로

값은 하나도 잃지 않는다. 잃는 것은 자리 관계다



격자 · 가로 세로가 있음

(1, 28, 28)

$1 \times 28 \times 28 =$

784

오늘 하루 내내 따라다니는 숫자

배치가 있으면 (32, 1, 28, 28) → (32, 784) · 맨 앞 축은 건드리지 않음

$$1 \times 28 \times 28 = 784$$

오늘 하루 내내 따라다니는 숫자

편다는 것

- 값은 하나도 사라지지 않음
- **자리 관계만 사라짐**
- 위아래로 붙어 있던 두 자리가 한 줄에서는 **28칸** 떨어짐

“ 격자에서 이웃이던 것이 한 줄에서도 이웃인 것은 아님 ”

퍼기에서 자주 나는 사고

x	(32, 1, 28, 28)	
nn.Flatten()(x)	(32, 784)	배치 축 보존
torch.flatten(x)	(25088,)	전부 뭉개짐
torch.flatten(x, 1)	(32, 784)	배치 축 보존

`nn.Flatten` 은 첫 축을 남기는 것이 기본

`torch.flatten` 은 아무것도 남기지 않는 것이 기본

다음 질문

층 하나 안에는

숫자가 몇 개 들어 있는가?

Linear 한 층은 곱셈표 하나

입력 자리마다 가중치 하나. 출력 칸마다 그 표가 한 줄



weight 모양은 (out_features, in_features) — 쓰는 순서와 반대다

in_features 를 784가 아닌 값으로 적으면 곱셈이 성립하지 않는다

weight 모양 읽기

(out_features, in_features)

```
nn.Linear(784, 200)
```

weight (200, 784) 쓴 순서와 반대

bias (200,)

합계 $784 \times 200 + 200 = 157,000$

출력 칸 하나마다 784개짜리 줄 하나

숫자를 잘못 적으면

```
RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x784 and 28x10)
```

- 앞은 **들어온 것** 32×784
- 뒤는 **총이 기대한 것** 28×10
- 가운데 두 숫자가 같아야 함

“ 오류 문장이 두 숫자를 그대로 보여 줌 ”

다음 질문

층을 두 개 쌓으면

항상 두 층인가?

층을 두 개 쌓았는데 한 개가 되는 경우

사이에 꺾어 주는 것이 없으면 두 층은 한 층으로 접힌다

사이에 아무것도 없을 때

`Linear(4,3) → Linear(3,2)`

= `Linear(4,2)` 하나와 값이 같음

실행으로 확인: `allclose True`

사이에 ReLU가 있을 때

`Linear → ReLU → Linear`

한 층으로 접히지 않음

층을 쌓은 값을 함

출력



ReLU

실행 결과

입력	-2.0	-0.5	0.0	0.5	2.0
출력	0.0	0.0	0.0	0.5	2.0

값을 만들지 않는다. 음수만 잘라 낸다

이 자르기 하나 때문에 층이 층으로 남는다

사이에 꺾는 것이 없으면
두 층은 한 층으로 접힘

ReLU가 하는 일

입력	-2.0	-0.5	0.0	0.5	2.0
출력	0.0	0.0	0.0	0.5	2.0

- 음수를 0으로
- 양수는 그대로
- 새 값을 만들지 않음

이 자르기 하나 때문에 층이 층으로 남음

오늘의 모델

Flatten	$(32, 1, 28, 28) \rightarrow (32, 784)$
Linear	$784 \rightarrow 200$
ReLU	
Linear	$200 \rightarrow 10$

네 줄이 전부임

나온 10개는 무엇인가

(32, 10)

첫 샘플 [-0.008, 0.033, -0.078, 0.045, 0.023,
-0.111, -0.017, -0.013, 0.002, -0.107]

합계 -0.231

가장 큰 칸 3번 (0.045)

- 확률 아님. 합이 1이 아님
- 크기 순서만 뜻이 있는 점수
- 가장 큰 칸의 번호가 예측

“ 확률로 바꾸는 이야기는 다음 시간

다음 질문

같은 모델을

다르게 적용할 수 있는가?

같은 모델, 두 가지 적는 법

파라미터 수도 출력 모양도 같다. 다른 것은 적는 방식뿐

클래스로 적기

```
class LinearClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(784, 200)
        self.act1 = nn.ReLU()
        self.fc2 = nn.Linear(200, 10)

    def forward(self, x):
        x = self.flatten(x)
        x = self.act1(self.fc1(x))
        return self.fc2(x)
```

Sequential로 적기

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 200),
    nn.ReLU(),
    nn.Linear(200, 10),
)
```

적은 순서가 곧 지나가는 순서

forward 를 따로 적지 않음

한 줄로 이어지는 길만 적을 수 있음

순서를 내가 정한다 → 갈라지거나 되돌아가는 길도 적을 수 있음

무엇이 같은가

출력 (32, 10) · 파라미터 159,010개 · 같은 계산

적는 방식은 사람이 읽기 위한 것이지 모델의 성질이 아니다

두 표기가 같은 것

출력	(32, 10)
파라미터	159,010개
계산	같음

적는 방식은 사람이 읽기 위한 것
모델의 성질이 아님

어느 쪽을 언제

- Sequential — 한 줄로 쪽 이어지는 길일 때
- 클래스 — 갈라지거나 되돌아가는 길이 필요할 때

지금 모델은 한 줄이라 어느 쪽이든 됨

“ 2일차 마지막 모델은 되돌아가는 길이 있음 ”

마지막 질문

이 모델은

얼마나 큰가?

파라미터는 어디에 몰려 있는가

784 → 200 → 10 구성 · 합계 159,010개

첫 Linear



$784 \times 200 + 200$
157,000개 (98.7%)

둘째 Linear



$200 \times 10 + 10$
2,010개 (1.3%)

Flatten · ReLU

가중치 없음
0개

첫 층 하나가 전체의 98.7%를 차지한다

입력 자리 수 784가 그대로 곱해지기 때문

입력이 커지면 첫 층만 이렇게 커진다

28×28 흑백	입력 784	1층 파라미터 157,000
32×32 컬러	입력 3,072	1층 파라미터 614,600
224×224 컬러	입력 150,528	1층 파라미터 30,105,800

파라미터 세는 법

첫 Linear	$784 \times 200 + 200 = 157,000$	98.7%
둘째 Linear	$200 \times 10 + 10 = 2,010$	1.3%
Flatten · ReLU	0	
합계	159,010	

입력 자리 수가 그대로 곱해짐

오늘 정리

Flatten	값 유지, 자리 관계 버림
Linear	(out, in) 곱셈표 + bias
ReLU	음수 자르기
출력	확률 아닌 점수 10개
크기	첫 층에 몰려 있음

남겨 두는 것

옷 사진 한 장을 **오른쪽으로 한 칸** 옮겼더니
784자리 중 **453자리**의 값이 바뀜

사람 눈에는 같은 옷인데

이 모델에게도 같은 옷일까?

벌점을 만드는 쪽과 보폭을 정하는 쪽

CNN · 챕터 3

앞 시간에 얻은 것

$(32, 1, 28, 28) \rightarrow \text{모델} \rightarrow (32, 10)$

첫 샘플 $[0.043, -0.037, \dots, 0.183]$

합계 -0.231

10개는 나왔음

맞았는지 틀렸는지는 아직 아무도 말해 주지 않음

오늘의 질문

같은 기울기를 받고도

왜 다르게 움직이는가?

오늘은 두 덩어리

앞 절반	점수 10개 → 벌점 하나
뒤 절반	벌점 → 보폭

가운데를 나누는 선이 오늘의 핵심임

첫 번째 질문

점수 10개를

어떻게 숫자 하나로 만드는가?

점수 10개가 벌점 하나가 되기까지

실제 실행값 · 학습 전 무작위 상태의 첫 샘플

클래스	0	1	2	3	4	5	6	7	8	9
로짓	+0.043	-0.037	-0.015	-0.152	-0.010	+0.159	+0.223	-0.013	+0.067	+0.183
↓ softmax — 크기 순서는 그대로, 합만 1로										
확률	0.099	0.092	0.094	0.082	0.094	0.111	0.119	0.094	0.102	0.114

10개를 다 더하면 1.0 — 실행으로 확인

벌점은 정답 칸 하나만 본다

정답 = 9 · 그 칸 확률 0.114

벌점 = $-\ln(0.114) = 2.17$

나머지 9칸은 합이 1이라는 조건으로만 개입

정답 칸 확률과 벌점

$p = 0.9 \rightarrow -\ln p = 0.105$

$p = 0.5 \rightarrow -\ln p = 0.693$

$p = 0.1 \rightarrow -\ln p = 2.303$

$p = 0.01 \rightarrow -\ln p = 4.605$

softmax가 하는 일

- 전부 양수로 만들고
- 합이 1이 되게 나눔

로짓	[0.043, ..., 0.159, 0.223, ..., 0.183]
확률	[0.099, ..., 0.111, 0.119, ..., 0.114]

크기 순서는 바뀌지 않음 — 가장 큰 칸은 그대로 6번

“ 예측 번호만 필요하면 softmax 없이 로짓에서 바로 읽어도 됨 ”

벌점은 정답 칸 하나만 본다

정답 칸 확률	벌점
0.9	0.105
0.5	0.693
0.1	2.303
0.01	4.605

정답 칸 확률이 낮을수록 **가파르게** 커짐

학습 전 벌점이 2.3 근처인 이유

클래스 10개 · 아무것도 모르면 각 칸 확률 0.1

$$-\ln(0.1) = 2.3026$$

실제 첫 배치 값 = 2.2908

첫 loss가 이 근처가 아니면 어딘가 잘못 연결된 것

“ 학습이 되는지 보는 가장 싼 점검

”

다음 질문

그럼 모델 안에

softmax를 넣어야 하나?

CrossEntropyLoss가 요구하는 것

무엇을 넣어야 하고, 무엇을 넣으면 안 되는가

이렇게 넣는다

예측 모델이 낸 로짓 그대로

(32, 10) · float · 합이 1이 아님

정답 클래스 번호

(32,) · int64 · 예: 9, 0, 0, 3

손실 함수 안에서 log_softmax가 돈다

이렇게 넣으면 어긋난다

① 미리 softmax를 걸어서 넣기

값은 나오지만 두 번 눌린 값

② 정답을 one-hot 정수로 넣기

RuntimeError · 실수형을 기대함

①은 오류가 나지 않아서 더 위험하다

같은 배치, 같은 모델로 잰 실행값

로짓을 그대로 넣었을 때

2.2908

softmax를 미리 걸어서 넣었을 때

2.3011

차이가 작아서 눈치채기 어렵다. 학습이 느려질 뿐 멈추지 않는다

CrossEntropyLoss 가 받는 것

예측	로짓 그대로 (32, 10)
정답	클래스 번호 (32,) int64

손실 함수 안에서 이미 눌러 줌

모델 마지막에 Softmax 를 넣지 않음

두 번 누르면

로짓 그대로	2.2908
softmax 미리 걸어서	2.3011

- 오류가 나지 않음
- 학습이 멈추지도 않음
- **느려질 뿐이라 눈치채기 어려움**

“ 오류가 나는 실수보다 이런 실수가 오래 감

”

정답은 번호로

```
labels          tensor([9, 0, 0, 3, ...])  int64  
one-hot(int)    RuntimeError
```

```
Expected floating point type for target  
with class probabilities, got Long
```

정수 번호를 주면 됨. one-hot으로 바꾸지 않음

잠깐, 다른 이름들

`BCELoss` 는

무엇이 다른가?

둘 중 하나를 고르는 문제에서 보게 될 이름들

오늘은 쓰지 않는다. 코드에서 보면 무엇인지 알아보는 정도

BCEWithLogitsLoss

로짓을 그대로 받음

sigmoid가 안에 있음

같은 예에서

0.2201

BCELoss

0~1 확률을 받음

sigmoid를 밖에서 걸어야 함

같은 예에서

0.2201

CrossEntropyLoss

칸이 2개인 로짓을 받음

클래스가 2개인 경우로 봄

같은 예에서

0.2201

예: 로짓 [2.0, -1.0] · 정답 [1, 0]

$\text{sigmoid}(2.0) = 0.8808$ · $\text{sigmoid}(-1.0) = 0.2689$

BCELoss에 로짓을 넣으면 오류가 난다

RuntimeError: all elements of input should be between 0 and 1

이름에 WithLogits가 붙어 있으면 sigmoid를 밖에서 걸지 않는다

CrossEntropyLoss 칸(클래스)가 2개만이 아님, 3개,
4개 그 이상 가능

이름 구분하는 법

이름	받는 것
<code>BCEWithLogitsLoss</code>	로짓
<code>BCELoss</code>	0~1 확률
<code>CrossEntropyLoss</code>	칸 여러 개인 로짓

`WithLogits` 가 붙어 있으면 밖에서 누르지 않음

오늘 쓰는 것

옷 10가지 중 하나를 고르는 문제 → `CrossEntropyLoss`

BCE 계열은 코드에서 보면 알아보는 정도로 충분함

“ 여기까지가 손실 함수. 이제 반대편으로 넘어감

”

두 번째 덩어리

벌점을 얻었다

값은 누가 바꾸는가?

누가 무엇을 건드리는가

기울기를 계산하는 쪽과 값을 바꾸는 쪽은 다르다

```
loss.backward()
```

기울기를 계산해서
.grad 를 채운다

파라미터 값은
아직 그대로



```
optimizer.step()
```

.grad 를 읽어서
파라미터 값을 바꾼다

기울기는
건드리지 않음



```
optimizer.zero_grad()
```

다음 배치를 위해
.grad 를 비운다

기본값은 0이 아니라
None

한 배치에서 실제로 댄 값

backward 전 .grad

None

backward 후 .grad 절댓값 평균

0.001810

backward 직후 파라미터가 그대로인가

True

SGD(lr=0.1) step 뒤 바뀐 양의 평균

0.000181

zero_grad 없이 backward를 한 번 더

0.001810 → 0.003183

$0.1 \times 0.001810 = 0.000181$ — 보폭은 기울기에 학습률을 곱한 만큼이었다

경계 하나만 기억하기

```
loss.backward()    기울기를 계산해서 .grad 를 채움  
optimizer.step()  .grad 를 읽어 파라미터를 바꿈
```

- `backward` 는 값을 **바꾸지 않음**
- `step` 은 기울기를 **다시 계산하지 않음**

한 배치에서 실제로 잔 값

backward 전 .grad	None
backward 후 .grad 절댓값 평균	0.001810
backward 직후 파라미터가 그대로	True
SGD(lr=0.1) step 뒤 바뀐 양 평균	0.000181

$$0.1 \times 0.001810 = 0.000181$$

보폭 = 기울기 × 학습률

zero_grad 를 빼면

backward 1회	0.001810
backward 2회	0.003183

기울기는 **덮어쓰이지** 않고 더해짐

`optimizer.zero_grad()` 기본은 0이 아니라 None으로 되돌림

한 배치에서 도는 다섯 줄

```
logits = model(images)
loss   = loss_fn(logits, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

zero_grad 는 backward 앞이면 어디든 됨
step 뒤로 가면 안 됨

다음 질문

같은 기울기를 받는데

왜 옵티마이저마다 다른가?

옵티마이저는 하나의 질문에 대한 네 가지 대답

같은 기울기를 받고도 보폭을 어떻게 정하는가

보폭을 어떻게 조절하는가

SGD	받은 기울기에 학습률을 곱한 만큼	언제나 같은 비율
Momentum	이전 방향을 얼마나 이어갈까	같은 방향이면 보폭이 커진다
AdaGrad	많이 움직인 방향은 줄이자	→ 너무 줄어드는 문제
RMSprop	최근 것만 보자	→ AdaGrad의 수정
Adam	둘을 합친 것	방향 잇기 + 방향별 크기 조절

네 개를 외우는 것이 목적이 아니다. 왜 하나로 끝나지 않았는지가 목적이다

첫 번째 대답

이전 방향을

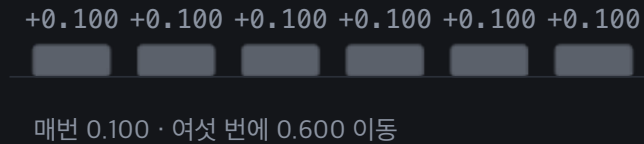
얼마나 이어갈까?

이전 방향을 이어가면 보폭이 어떻게 달라지는가

학습률 0.1 고정 · 같은 기울기 수열을 두 규칙에 똑같이 먹인 실측

① 같은 방향 기울기가 여섯 번 들어올 때

SGD

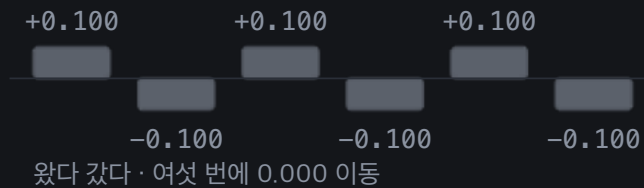


Momentum 0.9

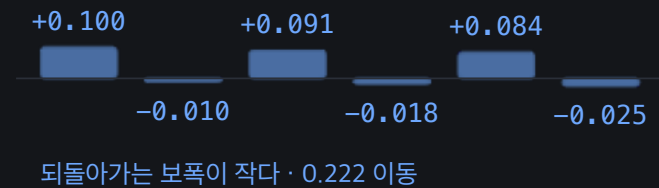


② 방향이 매번 뒤집힐 때

SGD



Momentum 0.9



이어가는 것이 이득이 될 때와 지나칠 때가 둘 다 있다

같은 방향이 반복되면 빨라지고, 방향이 흔들리면 되돌아가는 힘이 약해진다

Momentum이 바뀐 것

같은 방향 6회 0.100 → 0.469 누적 0.600 vs 1.783
방향이 뒤집힐 때 되돌아가는 보폭 -0.100 → -0.010

- 같은 방향이 반복되면 **보폭이 커짐**
- 방향이 흔들리면 **되돌아가는 힘이 약해짐**

“ 커지는 것이 언제나 이득은 아님. 지나칠 수도 있음 ”

다음 대답

많이 움직인 방향은
줄여도 되지 않을까?

많이 움직인 방향의 보폭을 줄이면 무슨 일이 생기는가

학습률 0.1 고정 · 기울기 5.0이 네 번 온 뒤 0.5로 작아지는 수열

보폭



AdaGrad

받은 기울기 제곱을 처음부터 전부 더한다

$0.100 \rightarrow 0.071 \rightarrow 0.058 \rightarrow 0.050 \dots$

더한 값은 줄어들지 않는다

→ 한 번 작아진 보폭이 회복되지 않는다

RMSprop

최근 것에 무게를 두고 옛것은 잊는다

$0.0179 \rightarrow 0.0189 \rightarrow \dots \rightarrow 0.0231$

기울기가 작아지면 보폭이 돌아온다

→ AdaGrad가 굳는 문제에 대한 수정

Adam — 방향 잇기(Momentum)와 방향별 크기 조절(RMSprop 쪽)을 함께 쓴다

같은 수열에서 Adam의 보폭 $0.100 \rightarrow 0.055$ · 줄지만 굳지 않는다

줄이는 방식 세 가지

AdaGrad	처음부터 전부 더함 → 굳음
RMSprop	최근 것만 봄 → 돌아옴
Adam	방향 잇기 + 방향별 크기 조절

기울기가 작아진 뒤 보폭

AdaGrad 0.0050 고정 RMSprop 0.0179 → 0.0231

오늘 정리

벌점	정답 칸 확률 하나로
계약	로짓 + 클래스 번호
경계	<code>backward</code> 계산 · <code>step</code> 갱신
옵티마이저	보폭 조절 방식의 차이

“ 오늘 숫자는 한 번의 통제된 실행값임. 어느 하나가 늘 낫다는 뜻 아님 ”

남는 질문

한 배치를 열 번 돌려 벌점을 2.29에서 0.23까지 내렸음
같은 배치를 계속 본 결과임

6만 장을 돌리면

무엇을 보고 잘됐다고 하나?

돌리고, 채점하고, 예측하기

CNN · 챕터 4

앞 시간에 만든 다섯 줄

```
logits = model(images)
loss   = loss_fn(logits, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

배치 하나를 처리하는 법은 다 나왔음

오늘의 질문

학습에 쓴 데이터로 채점하면

무엇을 못 보게 되는가?

오늘 지나갈 것

나누기	어떤 데이터로 채점할 것인가
돌리기	학습 루프와 검증 루프
구분하기	모드와 기울기 기록
쓰기	학습된 모델로 한 장 예측

먼저 이 질문

공부한 문제로 시험을 보면

점수는 무엇을 말해 주는가?

공부한 문제로 시험을 보면 무엇을 못 보게 되는가

데이터를 나누기 전과 나눈 뒤

나누지 않았을 때

점수가 높다는 것이
무슨 뜻인지 알 수 없다

가진 데이터 전부

→ 학습에 쓰고

→ 채점에도 쓴다

나누었을 때

train

학습에만 쓴다

파라미터가 이걸 보고 바뀐다

dev

고르는 데 쓴다

epoch 수·학습률·크기를 정한다

final holdout

마지막에 한 번

반복 선택에 쓰지 않는다

학습에 쓴 데이터로 낸 점수는 실력이 아니라 외운 정도까지 같이 낸 값이다

안 본 데이터로 재야 처음 보는 것에 대해 무엇을 할 수 있는지가 보인다

학습에 쓴 데이터로 낸 점수는
외운 정도까지 같이 낸 값임

세 덩어리로 나누기

train	파라미터가 이걸 보고 바뀜
dev	몇 epoch 돌릴지, 얼마나 크게 할지 정함
final holdout	마지막에 한 번

final holdout은 반복 선택에 쓰지 않음

왜 dev만으로는 부족한가

같은 dev로 후보 12개를 재고 가장 높은 것을 고르면

dev 정확도 12개	0.729 ~ 0.796	폭 6.7%p
1,000건에서의 흔들림	약 $\pm 2.5\%$	

고르는 데 쓴 자를 성능 보고에 그대로 쓰면
고른 만큼 유리하게 나올 여지가 생김

오늘 쓰는 분할

train	4,000건	학습
dev	1,000건	매 epoch 채점

$4000 \div 64 = 62 \dots 32 \rightarrow$ 배치 63개

수업 시간에 맞추려고 일부만 씀

다음 질문

채점할 때의 코드는

학습할 때와 어디가 다른가?

학습 루프와 검증 루프 — 없는 세 줄이 전부다

같은 배치 반복문 위에서 무엇이 있고 무엇이 없는가

학습 루프

```
model.train()
for images, labels in loader:
    logits = model(images)
    loss = loss_fn(logits, labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    총계에 더한다
```

검증 루프

```
model.eval()
with torch.no_grad():
    for images, labels in loader:
        logits = model(images)
        loss = loss_fn(logits, labels)
```

이 세 줄이 없다

총계에 더한다

실제로 돌린 결과 · 학습 4,000건 / 검증 1,000건

epoch	train loss	train acc	dev loss	dev acc
1	1.0705	0.6328	0.6877	0.7470
2	0.6400	0.7728	0.5841	0.8040
3	0.5539	0.8115	0.5593	0.8000

두 줄이 함께 내려간다 — 지금은 외우는 것보다 배우는 것이 앞서 있다

없는 세 줄

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

검증 루프에는 **파라미터를 바꾸는 줄이 하나도 없음**

“ 이 세 줄이 남아 있으면 그건 채점이 아니라 학습임 ”

3 epoch 결과

epoch	train loss	train acc	dev loss	dev acc
1	1.0705	0.6328	0.6877	0.7470
2	0.6400	0.7728	0.5841	0.8040
3	0.5539	0.8115	0.5593	0.8000

- 두 줄이 함께 내려감
- 1 epoch에서 dev가 더 높음 — **이상한 일 아님**

“ epoch 평균은 돌면서 좋아진 값의 평균이고, dev는 끝난 뒤 한 번 잰 값임 ”

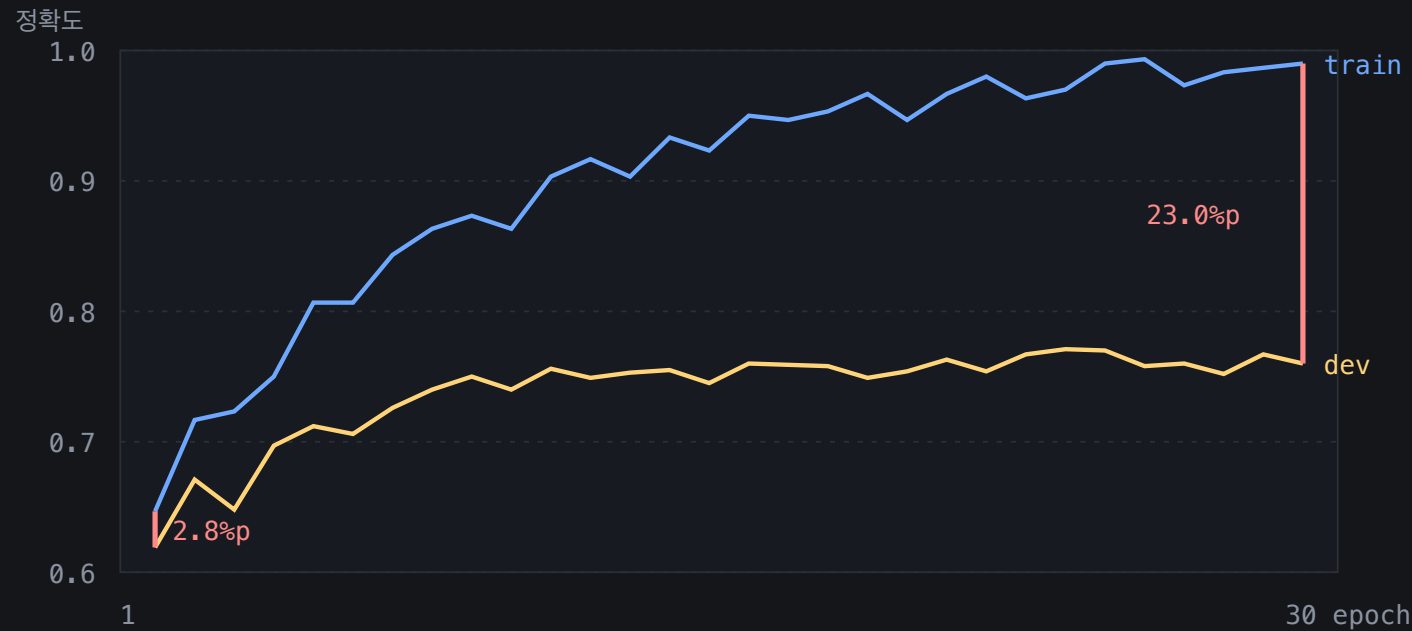
다음 질문

그럼 오래 돌릴수록

좋아지는가?

오래 돌리면 두 점수는 어떻게 갈라지는가

학습 300건 · 검증 1,000건 · 30 epoch 실측



무슨 일이 있었나

train 정확도

0.647 → 0.990

dev 정확도

0.619 → 0.760

간격

2.8%p → 23.0%p

train loss는 계속 내려감

train 점수만 보면 이 모델은 거의 다 맞는 모델이다

dev 점수를 같이 보면 열 문제 중 두 개 이상을 여전히 틀린다

dev 손실이 가장 낮았던 지점은 25 epoch (0.6498) · 30 epoch에서는 0.6880

이 한 번의 실행에서 관찰한 값이다. 항상 25에서 최저라는 뜻이 아니다

30 epoch 동안 벌어진 것

train	정확도	0.647 → 0.990
dev	정확도	0.619 → 0.760
간격		2.8%p → 23.0%p

train 쪽은 계속 좋아짐

dev 쪽은 어느 지점부터 거의 평평해짐

이 상태를 부르는 이름

과적합

학습 데이터에 **맞추는 일**은 계속 되고 있음
처음 보는 데이터에 대해서는 **더 나아지지 않음**

“ 오늘은 이름과 증상까지. 다루는 방법은 2일차에

”

그래서 dev로 무엇을 고르나

dev 손실 최저	25 epoch	0.6498
30 epoch		0.6880

- 몇 epoch에서 멈출지
- 층을 얼마나 크게 할지
- 학습률을 얼마로 할지

“ 이 한 번의 실행에서 25였음. 늘 25라는 뜻 아님

”

다음 질문

`eval()` 을 부르면

기울기도 안 쌓이는가?

train() / eval() 과 no_grad() 는 다른 축이다

하나는 층의 동작 모드, 하나는 기울기를 기록할지 말지

gradient 기록

	켜짐 (기본)	꺼짐 (no_grad)
train()	학습할 때 쓰는 칸 Dropout 무작위로 동작 기울기 기록함	기울기 없이 학습? 파라미터를 못 바꿈 학습이 되지 않음
총 동작 모드		
eval()	평가할 때 잘못된 조합 층은 평가 모드지만 기울기를 계속 쌓음	평가할 때 쓰는 칸 Dropout 통과만 함 기울기 기록 안 함

실행으로 확인한 것

Dropout(p=0.5) · train 모드로 같은 입력 두 번

Dropout(p=0.5) · eval 모드로 같은 입력 두 번

eval() 만 부르고 no_grad 없이 통과

Dropout 없는 오늘 모델은 두 모드 출력이

매번 다른 결과

두 번 다 같은 결과

출력에 grad_fn 이 그대로 남음

같음 · allclose True

eval() 은 no_grad() 를 대신하지 않는다. 반대로 마찬가지다

서로 다른 두 스위치

<code>train()</code> / <code>eval()</code>	층의 동작 모드
<code>no_grad()</code>	기울기 기록 여부

`eval()` 만 부르면 → 출력에 `grad_fn` 이 그대로 남음

하나가 다른 하나를 대신하지 않음

모드가 실제로 다른 층

Dropout($p=0.5$) · 같은 입력 두 번

train 모드 매번 다른 결과

eval 모드 두 번 다 같은 결과

오늘 모델에는 이런 층이 없어서 두 모드 출력이 같음
그래도 습관으로 붙여 둬 — 2일차에 이 층이 들어옴

어디에서 계산할 것인가

```
device = torch.device(  
    "cuda" if torch.cuda.is_available()  
    else "mps" if torch.backends.mps.is_available()  
    else "cpu")
```

- 모델과 데이터가 **같은 곳**에 있어야 함
- 없으면 `cpu` 로 떨어짐 — 느릴 뿐 돌아감

“ `cuda` 만 적어 둔 코드는 다른 기계에서 그대로 깨짐 ”

학습된 모델로 예측하기

예측 [9, 2, 1, 1, 6, 1, 4, 6, 5, 7, 2, 9]
정답 [9, 2, 1, 1, 6, 1, 4, 6, 5, 7, 4, 5]

이 배치 정확도 100 / 128

```
model.eval()  
with torch.no_grad():  
    pred = model(images).argmax(-1)
```

한 장만 넣을 때

(1, 28, 28) 그대로 넣어도 값이 나옴 - 흑백이라 우연히 맞음
(3, 32, 32) 컬러에서는 3장짜리 배치로 잘못 읽힘

`unsqueeze(0)` 으로 배치 축을 붙이기

그런데 이 모델,

같은 옷을 한 칸 옮기면 어떻게 될까?

만든 모델을 부순다

CNN · 챕터 5

오늘까지 온 길

챕터	무엇
1	이미지를 숫자 격자로 읽음
2	픽셀을 펴서 Linear에 넣음
3 · 4	그 모델을 학습시키고 채점함
5	그 모델의 한계를 확인함

지금까지 만든 것을

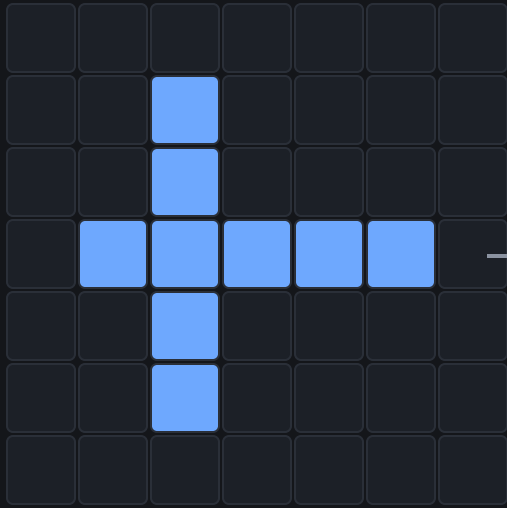
오늘 무너뜨림

무너뜨릴 근거를 숫자로 확보하는 것이 오늘의 일

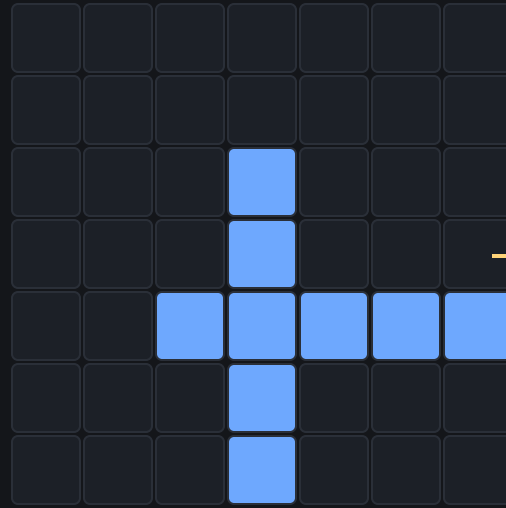
오늘의 질문

같은 모양이 위치만 바뀌면
무엇을 다시 배워야 하는가?

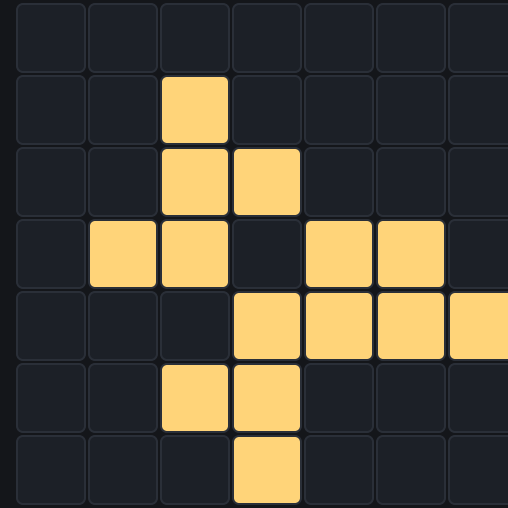
오전에 남겨 둔 질문으로 돌아온다



원래 십자 · 10이 9개



한 칸 옮긴 십자 · 10이 9개



값이 어긋난 자리 · 14개

사람 눈 — 같은 십자. 숫자 표 — 49자리 중 14자리가 어긋남.

오전에는 여기서 멈췄음. 지금부터 답한다.

오전에 멈춘 지점

- 사람 눈에는 **같은 십자**
- 숫자 표로는 49자리 중 **14자리**가 어긋남
- 모델이 이 둘을 같게 볼지 다르게 볼지는 답하지 않음

오늘 답함

먼저 확인할 것

우리가 만든 모델은

이 둘을 어떻게 받는가?

챕터 2에서 만든 구조

```
nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(784, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10),  
)
```

첫 줄이 오늘의 문제 지점

Flatten 이 하는 일

(1, 28, 28) → (784,)
격자 한 줄

- 값 784개는 그대로 남음
- 어느 값이 어느 값 옆에 있었는지는 남지 않음

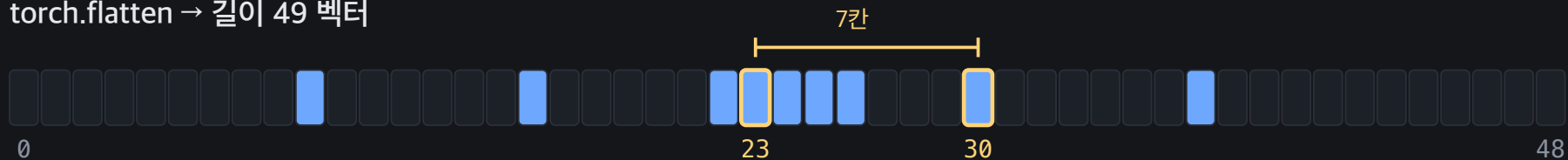
일렬로 펴는 순간 '이웃'이 사라진다

0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31	32	33	34
35	36	37	38	39	40	41
42	43	44	45	46	47	48

7×7 격자 · 각 칸의 평탄화 번호

23번과 30번은 위아래로 딱 붙어 있음

`torch.flatten` → 길이 49 벡터



격자에서의 거리 · 한 칸

일렬로 편 뒤의 거리 · 일곱 칸

가로 이웃은 1칸, 세로 이웃은 7칸 — 축마다 거리가 달라짐

7×7 이라 7칸. 28×28 이면 28칸 떨어짐

이웃이라는 사실은 번호 어디에도 적혀 있지 않음

모델이 받는 것은 번호가 붙은 값 49개뿐. 어느 값이 어느 값 옆에 있었는지는 넘어가지 않음.

이웃은 번호에 적혀 있지 않음

- 격자에서 위아래로 붙은 두 칸 → 번호로는 **7칸 떨어짐**
- 28×28 이면 **28칸 떨어짐**
- 가까움과 멀이 번호 어디에도 표시되지 않음

“ 모델이 이웃 관계를 무시하는 것이 아니라, 받지 못한 것임 ”

확인해 볼 것

픽셀 순서를 **고정된 방식으로 통째로 섞은** 입력을 넣어 봄

원본 배치	(4, 1, 28, 28)
픽셀을 섞은 배치	(4, 1, 28, 28)
평탄화 결과	(4, 784) - 길이 같음

모델의 층 구성과 파라미터 수가 달라지는지 봄

결과가 뜻하는 것

- 구조도 파라미터 수도 **그대로**
- 사람 눈에는 알아볼 수 없게 망가진 이미지
- 모델 쪽에서는 **아무 문제가 없는 입력**

“ 뜻하지 않는 것 — 섞어도 정확도가 같다는 뜻은 아님
여기서 본 것은 구조가 픽셀 순서를 전제하지 않는다는 사실 하나 ”

그러면 십자로 돌아가서

한 칸 옮긴 십자는

번호표에서 어떻게 보이는가?

같은 모양, 완전히 다른 번호표

0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31	32	33	34
35	36	37	38	39	40	41
42	43	44	45	46	47	48

원래 십자 · 10이 있는 번호

0	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31	32	33	34
35	36	37	38	39	40	41
42	43	44	45	46	47	48

한 칸 옮긴 십자 · 10이 있는 번호

10이 놓인 번호를 그대로 적으면

원래 9 16 22 23 24 25 26 30 37

옮김 17 24 30 31 32 33 34 38 45

겹치는 번호는 24, 30 두 개뿐

아홉 자리 중 일곱 자리가 처음 보는 번호

모델에게는 사실상 다른 입력

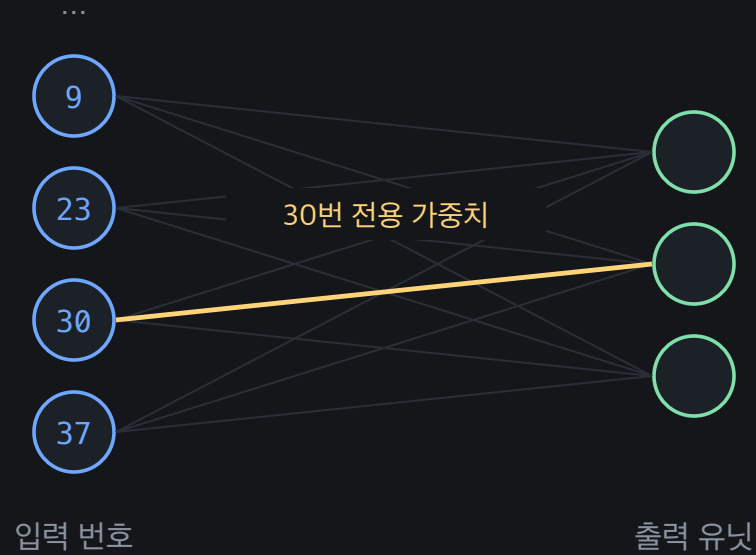
모양이 같다는 것은 사람의 판단. 번호표만 받는 쪽에서는 근거가 없음.

겹치는 번호는 두 개

원래	9	16	22	23	24	25	26	30	37
옮김	17	24	30	31	32	33	34	38	45

- 아홉 자리 중 **일곱 자리가 처음 보는 번호**
- 사람에게는 같은 십자, 번호표로는 거의 다른 입력

Dense 층은 '번호마다' 가중치를 따로 가진다



30번에서 배운 가중치는

31번에 쓰이지 **않음**

십자가 한 칸 옮겨 가면

새 번호의 가중치를 처음부터 다시

모양은 같아도 배움이 옮겨 가지 **않음**

위치가 49가지면 49번 따로 겪어야 함

Dense는 '무엇이 보이는가'가 아니라 '몇 번 자리에 값이 있는가'로 배움.

같은 모양 + 다른 위치

→ 다른 번호

→ 다른 가중치

배운 것이 옮겨 가지 않음

이동 실험

한 픽셀만 옆으로 민 이미지를 같은 모델에 넣음

이동 전후 이미지 차이(평균 절댓값)
평탄화 벡터 코사인 유사도
Linear 출력의 변화량

세 숫자를 같이 봄

첫 번째 진단

위치가 바뀌면 처음부터 다시 배워야 함

- 십자를 왼쪽 위에서 배워도
- 오른쪽 아래 십자는 다른 문제
- 위치가 많을수록 겪어야 할 경우도 늘어남

문제가 하나 더 있음

입력이 커지면

가중치는 몇 개가 되는가?

가중치를 실제로 세어 보면

torch 로 확인한 값 · Linear(784, 128) 과 Conv2d(1, 8, 3)

Linear(784 → 128) 100,352개

= 784×128

Conv2d(1 → 8, 3×3) 72개

= $8 \times (1 \times 3 \times 3)$

같은 자리에 놓고 비교하면 약 1,394배

bias 까지 세면 100,480개 대 80개

입력이 커지면 어떻게 되나

입력	퍼면	Linear(→128) 가중치	Conv 파라미터
(1, 28, 28)	784	100,352	80
(3, 32, 32)	3,072	393,216	224
(3, 224, 224)	150,528	19,267,584	224

두 계산을 나란히

Linear(784 → 128)	784×128	= 100,352
Conv2d(1 → 8, 3×3)	$8 \times (1 \times 3 \times 3)$	= 72

- 앞은 **입력 칸 수 × 출력 칸 수**
- 뒤는 **필터 수 × 창 하나의 크기**

약 **1,394배** 차이

입력을 키우면

입력	퍼면	Linear(→128)	Conv2d(→8, 3)
(1, 28, 28)	784	100,352	80
(3, 32, 32)	3,072	393,216	224
(3, 224, 224)	150,528	19,267,584	224

Conv 쪽은 224×224가 되어도 **숫자가 그대로**

두 번째 진단

- Linear 첫 층 — 가중치가 **입력 면적에 비례**
- Conv — 가중치가 **입력 면적과 무관**

“ 파라미터가 적다는 것과 성능이 좋다는 것은 다른 이야기임
여기서 본 것은 개수 하나

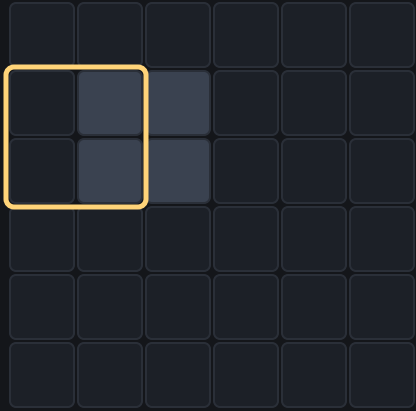
”

그러면

무엇을 어떻게
바꿔야 하는가?

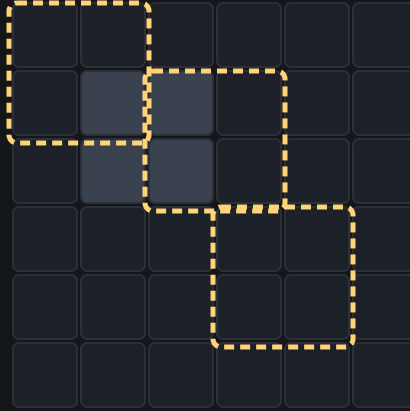
바뀌야 할 것은 두 가지

① 한 번에 좁게 본다



한 칸이 아니라 붙어 있는 작은 구역을 한 덩어리로
가까운 자리끼리 관계가 있다는 가정

② 같은 창을 옮겨 가며 쓴다



창 안의 숫자표는 한 벌뿐
자리를 옮겨도 같은 숫자표를 다시 씀

이 두 가지가 붙으면

- 가중치가 자리 수에 안 묶임
- 한 자리에서 배운 것이
다른 자리에도 그대로 쓰임

위치가 바뀌어도

다시 배우지 않아도 됨

창 안에서 어떤 계산을 하는지는 다음 시간

① 한 번에 좁게 보기

② 같은 창을 옮겨 가며 쓰기

이 둘이 붙으면 위치가 바뀌어도 다시 배우지 않아도 됨

움기면 결과도 같이 옮겨짐

작은 점 하나를 찍은 이미지와, 그 이미지를 한 칸 민 이미지

피쳐맵을 한 칸 되돌리면 원래 피쳐맵과 같은가

가장자리를 뺀 안쪽에서 확인함

여기서 자주 생기는 오해 둘

× 작은 구역만 보니까
넓은 것은 못 본다

한 층은 좁게 봄

그 결과를 다시 좁게 보면

원본에서는 더 넓은 구역이 됨

넓게 보는 방법이 층을 쌓는 것

얼마나 넓어지는지는 크기 계산에서 다룸

× 창을 옮길 때마다
숫자표를 복사한다

복사가 아니라 같은 한 벌

자리를 옮겨도 파라미터는

한 개도 늘지 않음

28×28 이든 224×224 이든

Conv2d(1, 8, 3) 파라미터는 80으로 같음

오늘 정리

펴기	값은 남고 이웃은 사라짐
위치 이동	번호가 바뀌고 가중치도 바뀜
크기	Linear는 면적에 비례, Conv는 무관
필요한 것	좁게 보기 + 창 공유

아직 답하지 않은 것

창 안의 값 9개 → ? → 숫자 하나

창 하나가 숫자 하나를

어떻게 만드는가?

창 안에서 벌어지는 일

CNN · 챕터 6

앞 시간이 남긴 것

- 펴면 이웃이 사라짐
- 위치가 바뀌면 가중치가 바뀜
- 그래서 **좁게 보고, 같은 창을 옮겨 쓰기로 함**

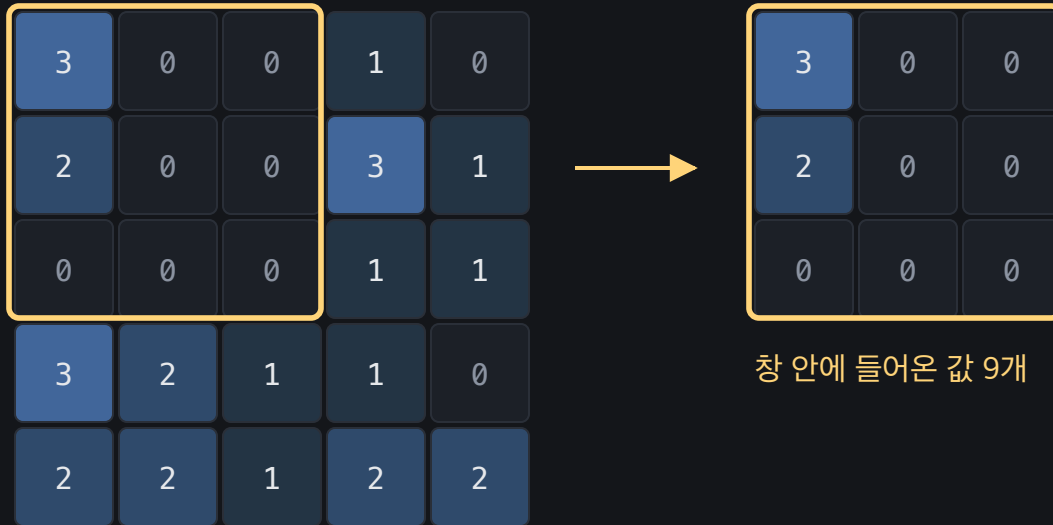
창 안에서 무슨 계산을 하는지는 비워 둠

오늘의 질문

작은 창 하나가

숫자 하나를 어떻게 만드는가?

창 하나를 격자 위에 얹는다



창 안에 들어온 값 9개

한 번에 보는 것은

전체 25개가 아니라 9개

이 9개로 만드는 것은

숫자 하나

9 → 1. 어떻게 하나로 줄이는가

입력 5×5 · 값은 밝기

창을 옮겨 가며 이 과정을 반복하면 숫자들이 다시 격자를 이룬다.

재료는 두 개

입력 창	이미지에서 잘라 온 값 9개
커널	우리가 들고 다니는 숫자 9개

크기가 같아야 서로 겹쳐 놓을 수 있음

규칙은 한 줄

같은 자리끼리 곱해서

전부 더한다

곱셈 9번, 덧셈 8번. 그게 전부임

출력 한 칸을 손으로 계산한다

같은 자리끼리 곱해서 전부 더한다



$$\begin{aligned} & 3 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) + 2 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) + 0 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) \\ &= 3 + 0 + 0 + 2 + 0 + 0 + 0 + 0 + 0 = 5 \end{aligned}$$

커널 가운데 열이 0이므로, 이 커널은 결국

$$\text{왼쪽 열의 합} - \text{오른쪽 열의 합} = (3+2+0) - (0+0+0) = 5$$

숫자가 어디서 왔는지

$$\begin{array}{rcl} 3 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) & = & 3 \\ 2 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) & = & 2 \\ 0 \cdot 1 + 0 \cdot 0 + 0 \cdot (-1) & = & 0 \\ \text{합} & = & 5 \end{array}$$

만들어 낸 값이 아니라 **아홉 번의 곱셈이 남긴 값**

이 커널이 재는 것

가운데 열이 전부 0

$$\begin{array}{rcl} \text{왼쪽 열의 합} & - & \text{오른쪽 열의 합} \\ (3+2+0) & - & (0+0+0) & = 5 \end{array}$$

왼쪽이 밝을수록 커지고, 오른쪽이 밝을수록 작아짐

커널 — 학습되는 것은 이 숫자 아홉 개

1	0	-1
1	0	-1
1	0	-1

3×3 커널 · 값 9개

Conv2d(1, 1, 3) 이면 여기에 bias 1개가 더

학습되는 것

- 창 안 숫자 9개
- bias 1개

기울기를 받아

이 값들이 바뀜

처음에는 무작위 값

학습되지 않는 것

- 창을 어디에 놓을지
- 창을 몇 칸씩 옮길지

그건 우리가

미리 정해 두는 규칙

다음 시간에 이 규칙을 다룸

지금 쓰는 이 커널은 설명하려고 우리가 직접 적은 값. 실제 학습에서는 값이 스스로 정해짐.

학습되는 것과 아닌 것

학습됨	학습 안 됨
커널 값 9개	창을 어디에 놓을지
bias 1개	창을 몇 칸씩 옮길지

지금 화면의 커널은 **설명하려고 우리가 적은 값**

한 칸을 구했으면

다음 칸은

어디서 나오는가?

창을 오른쪽으로 한 칸씩 옮긴다

맨 윗줄 세 번 · 세로 위치는 그대로

3	0	0	1	0
2	0	0	3	1
0	0	0	1	1
3	2	1	1	0
2	2	1	2	2

가로 1번째 자리

왼쪽 열 5 - 오른쪽 열 0

= 5



3	0	0	1	0
2	0	0	3	1
0	0	0	1	1
3	2	1	1	0
2	2	1	2	2

가로 2번째 자리

왼쪽 열 0 - 오른쪽 열 5

= -5



3	0	0	1	0
2	0	0	3	1
0	0	0	1	1
3	2	1	1	0
2	2	1	2	2

가로 3번째 자리

왼쪽 열 0 - 오른쪽 열 2

= -2

여기까지 나온 세 숫자가 출력의 첫 줄

5	-5	-2
---	----	----

첫 줄이 나옴

5 -5 -2

- 왼쪽이 밝던 자리 → 양수
- 오른쪽이 밝던 자리 → 음수
- 창을 옮겼을 뿐, 커널은 그대로

한 줄이 끝나면

어디로 가는가?

한 줄이 끝나면 아래로 내려간다

맨 왼쪽으로 돌아가서 다시 오른쪽으로

3	0	0	1	0
2	0	0	3	1
0	0	0	1	1
3	2	1	1	0
2	2	1	2	2

세로 1번째 자리

왼쪽 열 5 - 오른쪽 열 0

= 5

3	0	0	1	0
2	0	0	3	1
0	0	0	1	1
3	2	1	1	0
2	2	1	2	2

세로 2번째 자리

왼쪽 열 5 - 오른쪽 열 1

= 4

3	0	0	1	0
2	0	0	3	1
0	0	0	1	1
3	2	1	1	0
2	2	1	2	2

세로 3번째 자리

왼쪽 열 5 - 오른쪽 열 2

= 3

창은 가로 3자리 × 세로 3자리 = 9자리에 놓임. 그래서 출력도 3×3.

창의 숫자표는 아홉 번 모두 같은 것을 씀 — 매번 새로 만드는 것이 아님

창이 놓인 자리를 세면

가로 3자리 × 세로 3자리 = 9자리

- 계산 9번
- 결과 숫자 9개
- 그 9개가 다시 3×3 격자를 이룸

아홉 번 계산하면 격자가 하나 더 생긴다

3	0	0	1	0
2	0	0	3	1
0	0	0	1	1
3	2	1	1	0
2	2	1	2	2

입력 5×5

3×3 커널
→

5	-5	-2
4	-3	-1
3	0	-1

출력 3×3 · 피쳐맵

양수 — 왼쪽이 더 밝은 자리

음수 — 오른쪽이 더 밝은 자리

왜 5가 3이 되었나

창이 놓일 수 있는 자리를

세어 본 것뿐

$$5 - 3 + 1 = 3$$

가로도 세로도 같은 계산

한 축씩 따로 세는 것

일반 규칙은 다음 시간에

F.conv2d(x, kernel) 로 돌린 값과 손계산이 자리마다 전부 같음. 확인하고 넘어감.

이 격자가 피처맵

한 커널이 이미지 전체를 훑고 남긴 기록

5가 3이 된 이유

$$5 - 3 + 1 = 3$$

- 창이 놓일 수 있는 자리를 센 것뿐
- 가로도 세로도 같은 계산
- **한 축씩 따로** 셈

“ 일반 규칙은 다음 시간에 정리함 ”

손계산을 코드로 옮기면

```
for i in range(H_out):  
    for j in range(W_out):  
        patch = x[0, 0, i:i+3, j:j+3]  
        manual[i, j] = (patch * kernel[0, 0]).sum()
```

방금 아홉 번 한 일을 그대로 적은 것

같은 일을 라이브러리로

```
auto = F.conv2d(x, kernel, bias=None, stride=1, padding=0)
```

두 결과가 자리마다 같은지 확인함

일치가 뜻하는 것

- `conv2d` 가 하는 일은 **방금 손으로 한 것과 같음**
- 뜻하지 않는 것 — 속도나 구현 방식이 같다는 뜻은 아님

“ 블랙박스가 하나 줄어든 것

”

이름 정리

부르는 말	가리키는 것
커널	창 안의 숫자표
필터	보통 같은 것을 가리킴
피처맵	훑고 나온 결과 격자

부르는 말이 여럿이어도 물건은 하나

다음 질문

커널 값이 달라지면
무엇이 달라지는가?

같은 그림, 커널만 바꾸면

오른쪽 절반이 밝은 9×9 이미지 · 출력은 7×7

0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1

입력 · 세로 경계 하나

1	0	-1
1	0	-1
1	0	-1

좌우를 빼는 커널

↓

0	0	-3	-3	0	0	0
0	0	-3	-3	0	0	0

∴ (7줄 모두 같은 모습)

경계 자리에만 -3 이 뜬
나머지는 전부 0

1	1	1
0	0	0
-1	-1	-1

위아래를 빼는 커널

↓

0	0	0	0	0	0	0
0	0	0	0	0	0	0

∴ (7줄 모두 같은 모습)

같은 그림인데 전부 0
위아래로는 변화가 없어서

읽는 법

피쳐맵의 큰 값 =
그 자리에서 커널과
닮은 모습이 있었다

0 이라고 아무것도 없는 게
아니라, 이 커널이 찾는 것이
없다는 뜻

피처맵을 읽는 법

- 값이 큼 → 그 자리에서 **커널과 닮은 모습**이 있었음
- 값이 0 → 아무것도 없는 것이 아니라 **이 커널이 찾는 것이 없음**

같은 이미지라도 커널이 바뀌면 다른 곳이 켜짐

코드로 확인

```
vertical = torch.tensor([[1., 0., -1.]] * 3).view(1, 1, 3, 3)  
horizontal = vertical.transpose(2, 3).contiguous()
```

오른쪽 절반이 밝은 9×9 이미지에 각각 적용함

실제 사진에도

FashionMNIST 한 장 `(1, 1, 28, 28)` 에 같은 커널을 적용

출력 shape `(1, 1, 26, 26)`

세로 방향 경계와 가로 방향 경계가 서로 다른 자리에서 뜸

여기서 주의

- 지금 커널은 **우리가 직접 적은 값**
- 학습에서는 이 아홉 개가 **스스로 정해짐**
- 학습된 커널이 사람이 알아볼 모양을 만든다는 보장은 없음

“ 예시로 쓴 커널을 CNN이 실제로 배우는 것이라고 말하지 않음 ”

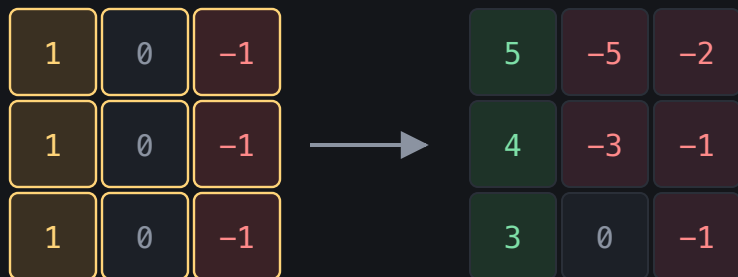
이름 이야기 하나

합성곱이라고 부르는데

정말 합성곱인가?

이름은 합성곱, 계산은 조금 다르다

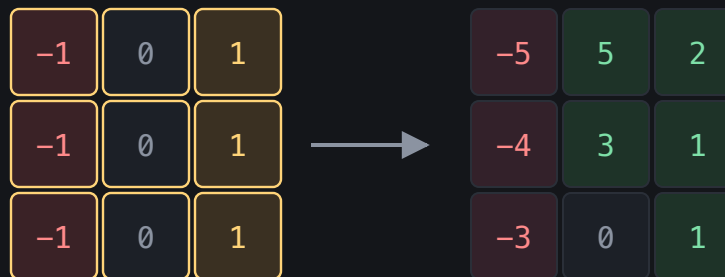
PyTorch 가 실제로 하는 것



커널을 그대로 얹고 곱해서 더함

우리가 손으로 계산한 그 값

수학 교과서의 합성곱



커널을 180° 돌려서 얹음

부호가 통째로 반대가 됨

엄밀히 말하면 PyTorch 의 conv2d 는 cross-correlation. 커널을 뒤집지 않음.

커널 값 자체가 학습되므로 결과 성능은 달라지지 않음. 다만 손으로 대조할 때 헛갈리지 않으려고 알아 둬.

경계

- `conv2d` 가 하는 계산은 엄밀히 **cross-correlation**
- 수학의 합성곱은 커널을 180° 돌려서 엮음
- 커널 값 자체가 학습되므로 **결과 성능은 달라지지 않음**

“ 손으로 대조할 때 부호가 뒤집혀 보이면 여기를 의심 ”

마지막 질문

창을 크게 하면

더 좋은 것을 뽑는가?

그러면 창을 크게 하면 더 좋은가

입력 (1, 28, 28) · 필터 8개 · torch 로 확인한 값

커널 크기	파라미터 수	출력 크기	한 번에 보는 자리
1×1	16	(8, 28, 28)	1
3×3	80	(8, 26, 26)	9
5×5	208	(8, 24, 24)	25
7×7	400	(8, 22, 22)	49

× 큰 커널이 자동으로 더 좋은 정보를 뽑는다

커널이 커지면 확실히 늘어나는 것 — 파라미터 수와 계산량

확실히 줄어드는 것 — 출력 격자의 크기

더 나은 표현이 나오는지 데이터와 구조에 달린 문제. 크기만으로 정해지지 않음.

커널 크기에 대한 경계

커널을 키우면 **확실한 것**

- 파라미터 수와 계산량이 늘어남
- 출력 격자가 작아짐

확실하지 않은 것

- 더 나은 표현이 나오는지 — 데이터와 구조에 달림

오늘 정리

계산	같은 자리끼리 곱해서 다 더함
커널	학습되는 숫자표
피처맵	창을 다 훑고 남은 격자
크기	$5 - 3 + 1 = 3$, 축마다 따로
경계	실제로는 cross-correlation

아직 정하지 않은 것

창을 한 칸씩? 두 칸씩?
가장자리는 어떻게?

창을 어떻게 움직이면

출력이 왜 달라지는가?

창을 어떻게 움직이는가

CNN · 챕터 7

앞 시간과 오늘

챕터 6	창 하나가 숫자 하나를 만드는 원리
챕터 7	창을 옮기는 규칙과 크기 계산

앞이 감이었다면 오늘은 **정확한 숫자**

오늘의 질문

창을 어떻게 움직이면

출력이 왜 달라지는가?

창을 몇 칸씩 옮길지는 우리가 정한다

가로 한 줄만 · 칸 7개 · 창 3칸

한 칸씩 옮기면

0	1	2	3	4	5	6	1번째
0	1	2	3	4	5	6	2번째
0	1	2	3	4	5	6	3번째
0	1	2	3	4	5	6	4번째
0	1	2	3	4	5	6	5번째

창이 놓인 자리 5개 → 결과도 5개

두 칸씩 옮기면

0	1	2	3	4	5	6	1번째
0	1	2	3	4	5	6	2번째
0	1	2	3	4	5	6	3번째

창이 놓인 자리 3개 → 결과도 3개

같은 입력, 같은 커널. 옮기는 폭만 바꿨는데 결과 개수가 5개와 3개로 같림.

건너뛴 자리에서는 계산 자체를 하지 않음. 계산을 나중에 버리는 것이 아님.

세어 본 결과

한 칸씩	→	창 5개	→	결과 5개
두 칸씩	→	창 3개	→	결과 3개

같은 입력, 같은 커널. **움기는 폭만 다름**

방금 그 '움기는 폭'에 이름을 붙인다

stride

창을 한 번에 몇 칸 옮기는가

stride=1 → 한 칸씩

stride=2 → 두 칸씩

2차원에서 같은 것을 하면 (입력 28×28 · 커널 3×3 · 패딩 없음)



stride 를 키우면

- 출력 격자가 작아짐
- 창이 놓이는 횟수가 줄어듦
- 중간 자리를 건너뛴

28÷2 = 14 가 아니라 13. 왜 그런지는 곧 보

stride

창을 한 번에 몇 칸 옮기는가

- 키우면 출력 격자가 작아짐
- 키우면 창을 놓는 횟수가 줄어듦
- 건너뛴 자리에서는 계산 자체를 하지 않음

여기서 걸리는 것

$$28 \div 2 = 14$$

그런데 `stride=2` 로 얻은 출력은 **13**

“ 왜 그런지는 조금 뒤에 봄 ”

다른 문제

가장자리 칸은

계산에 몇 번 들어가는가?

가장자리 칸은 계산에 몇 번 들어가는가

입력 5×5 · 커널 3×3 · 한 칸씩

1	2	3	2	1
2	4	6	4	2
3	6	9	6	3
2	4	6	4	2
1	2	3	2	1

각 칸이 창에 들어간 횟수
가운데 9번 · 모서리 1번

그래서 생기는 일

- 출력이 5에서 3으로
- 층을 쌓을수록 계속 줄어듦
- 가장자리 정보가 덜 반영됨

바깥에 한 줄 더 붙이면?

바깥을 0으로 한 줄 채우면

0	1	2	3	4	5	0
---	---	---	---	---	---	---

원래 값 5개 + 바깥 0 두 개 = 7칸

$$7 - 3 + 1 = 5$$

입력과 같은 크기로 나옴

이 '바깥 채우기'가 padding. 값은 보통 0 — 없는 자리를 만들어 준 것뿐이지 관측한 값이 아님.

padding

바깥에 칸을 더 붙이는 것

$$\begin{array}{l} 5\text{칸} + \text{양쪽에 한 칸} = 7\text{칸} \\ 7 - 3 + 1 = 5 \end{array}$$

입력과 같은 크기로 나옴

채워 넣는 값

- 보통 0
- 없는 자리를 만들어 준 것뿐
- 관측한 값이 아님

“ 가장자리 근처의 결과는 이 인공적인 0의 영향을 받음 ”

매번 세지 않으려면

출력 크기를

계산할 수 있는가?

출력 크기는 세는 문제다

창이 놓일 수 있는 왼쪽 끝 자리를 세면 끝

$$O = \text{floor}((I + 2P - D(K-1) - 1) / S + 1)$$

- I 입력 한 축의 크기
- K 커널 한 축의 크기
- S 옮기는 폭 (stride)
- P 한쪽에 붙인 칸 수 (padding)
- D 커널 원소 사이 간격 (dilation) · 기본 1

반드시 같이 기억할 것

① floor — 나누어떨어지지 않으면 내림

② 축마다 따로 — 가로와 세로를 각각

③ 2P — 양쪽에 붙이므로 두 배

D=1 이면 $D(K-1)+1 = K$. 우리가 쓰는 경우는 전부 D=1

$(I + 2P - D(K-1) - 1) / S + 1$ · I=28, K=3, S=1, P=0 → $(28 + 0 - 2 - 1)/1 + 1 = 26$

같은 식에 S=2 만 넣으면 → $(28 + 0 - 2 - 1)/2 + 1 = 13.5$ → 내림해서 13

공식의 뿌리

창의 **왼쪽 끝**이 놓일 수 있는 자리를 세는 것

$$0, \quad S, \quad 2S, \quad \dots \quad \leq \quad I + 2P - K$$

세어 보면 $(I + 2P - K) / S + 1$

반드시 같이 기억할 셋

1. **floor** — 나누어떨어지지 않으면 내림
2. **축마다 따로** — 가로와 세로를 각각
3. **2P** — 양쪽에 붙이므로 두 배

D는 무엇인가

$$0 = \text{floor}((I + 2P - D(K-1) - 1) / S + 1)$$

- D는 커널 원소 사이를 벌리는 값. 기본값 1
- $D = 1$ 이면 $D(K-1)+1 = K$
- 이 강의에서 쓰는 경우는 전부 $D = 1$

“ 이름만 알아 두고 넘어감. 오늘 다루지 않음 ”

공식과 실제를 대조

I	K	S	P	D	공식	실제	일치
---	---	---	---	---	----	----	----

여러 조합을 한 줄씩 확인함

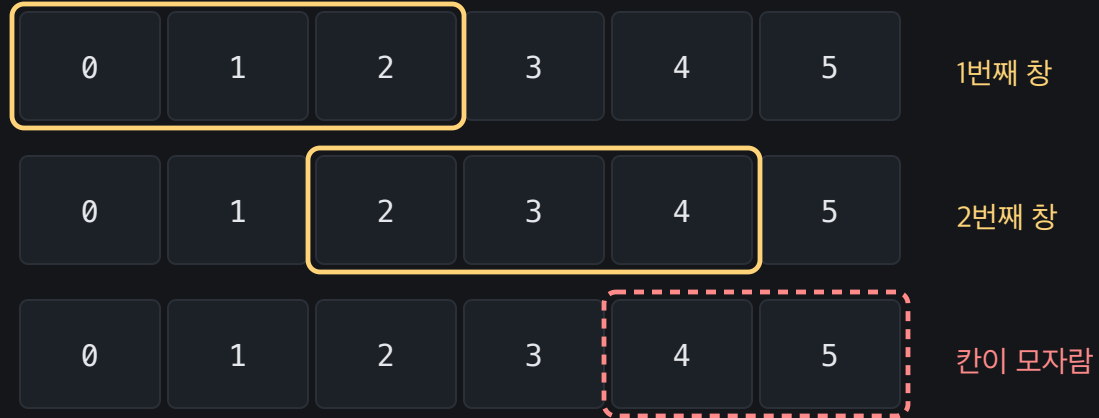
한 줄이라도 어긋나면 공식 쪽을 고쳐야 함

아까 걸렸던 것

창을 2.5개 놓을 수 있는가?

2.5개의 창은 없다

칸 6개 · 창 3칸 · 두 칸씩 옮김



4번 자리에서 시작하면 창 3칸을 채울 수 없음 → 놓지 않음

$$(6 + 0 - 1 \cdot (3-1) - 1) / 2 + 1 = 3/2 + 1 = 2.5 \rightarrow \text{floor} \rightarrow 2$$

2.5 는 예외가 아니라 floor 가 하는 일이 눈에 보이는 지점

마지막 칸 5번은 어떤 창에도 들어가지 못하고 버려짐

같은 일이 28×28 에서도 일어남 — K=3, S=2, P=0 이면 13.5 → 13, 가장자리 한 줄이 버려짐

2.5가 2가 되는 자리

$$(6 + 0 - 1 \cdot (3-1) - 1) / 2 + 1 = 2.5 \rightarrow 2$$

- 남는 칸에서 시작하면 창을 채울 수 없음 → 놓지 않음
- 마지막 칸은 어떤 창에도 들어가지 못하고 버려짐

예외가 아니라 내림이 눈에 보이는 지점

코드로 확인

```
(6 + 0 - 1*(3-1) - 1) / 2 + 1 = 2.5 -> 내림 2  
실제 출력 (1, 1, 2, 2)
```

padding=1 을 주면 어떻게 되는지도 같이 봄

크기를 지키고 싶다면

`stride = 1` 일 때

$$P = (K - 1) / 2$$

K	P	28 →
3	1	28
5	2	28
7	3	28

padding='same'

```
nn.Conv2d(1, 4, kernel_size=3, stride=1, padding='same')
```

- stride=1 일 때만 크기를 유지함
- stride=2 에서는 만들 때부터 오류

“ 편의 기능임. 공식을 대신하지 않음

”

실제로 나는 오류

```
ValueError: padding='same' is not supported  
for strided convolutions
```

무엇이 안 되는지를 그대로 말해 줌

자주 듣는 말

`stride=2` 로 바꾸면

계산이 절반이 되는가?

stride 2 로 바꾸면 계산이 절반이 되는가

입력 28×28 · 커널 3×3 · padding 1 · torch 로 확인

stride = 1

출력 28 × 28

창을 놓는 횟수 784

곱셈 $784 \times 9 = 7,056$

창 하나당 곱셈 9번

stride = 2

출력 14 × 14

창을 놓는 횟수 196

곱셈 $196 \times 9 = 1,764$

창 하나당 곱셈은 그대로 9번

× "stride 2 면 계산이 절반"

줄어드는 축이 가로와 세로 두 개. 각각 절반이면 넓이는 절반의 절반.

$1,764 / 7,056 = 0.25 \rightarrow$ 약 1/4

가로 한 줄만 보면 절반이 맞음. 2차원 격자 전체로는 약 1/4.

절반이 아님

stride 1	$28 \times 28 = 784$	창	곱셈 7,056
stride 2	$14 \times 14 = 196$	창	곱셈 1,764

$1,764 / 7,056 = 0.25$

2차원에서는 약 1/4

왜 1/4인가

- 줄어드는 축이 **가로와 세로 두 개**
- 각 축이 절반이면 넓이는 절반의 절반

“ 가로 한 줄만 보면 절반이 맞음
격자 전체로는 약 1/4 — 여기서 자주 틀림

”

다음 질문

가로와 세로가 다르면

어떻게 계산하는가?

가로와 세로는 각각 계산한다

정사각이 아니면 한 번에 뚫그릴 수 없음

입력

$H = 10$ $W = 7$

$\text{kernel} = (3, 5)$

세로 3칸 · 가로 5칸짜리 창

$\text{stride} = (2, 1)$

$\text{padding} = (1, 0)$

앞자리가 세로, 뒷자리가 가로

세로 축

$(10 + 2 \cdot 1 - (3 - 1) - 1) / 2 + 1 = 5.5$

→ floor → 5

가로 축

$(7 + 2 \cdot 0 - (5 - 1) - 1) / 1 + 1 = 3$

→ 3

출력 shape (1, 1, 5, 3) — torch 로 돌린 결과와 같음. 두 축을 각각 세었을 뿐.

정사각 입력에서는 두 계산이 같아서 한 번만 하는 것처럼 보임. 규칙 자체는 언제나 축별.

축마다 따로

```
kernel   = (3, 5)      앞이 세로, 뒤가 가로  
stride   = (2, 1)  
padding  = (1, 0)
```

```
(1, 1, 10, 7) → (1, 1, 5, 3)
```

정사각에서는 두 계산이 같아서 한 번만 하는 것처럼 보임

거꾸로 풀어 보기

입력 (1, 1, 28, 28) → Conv2d 한 번 → 출력 (1, 8, 14, 14)

커널은 7 이하, 패딩은 3 이하

무엇을 알 수 있는가?

거꾸로 — 출력만 보고 무엇을 했는지 맞추기

주어진 것

입력 (1, 1, 28, 28) → Conv2d 한 번 → 출력 (1, 8, 14, 14)

단, 커널은 7 이하 · 패딩은 3 이하로 둬 (범위를 정하지 않으면 답이 한없이 늘어남)

바로 읽히는 것

· 필터 개수 = 8 (출력 채널 자리)

· 28 → 14 로 줄었으므로 stride = 2

이 범위에서 stride=1 이면 아무리 커널을 키워도 22 아래로 못 내려감

stride=3 이면 14 에 닿지 않음

정해지지 않는 것

K=1 S=2 P=0 → 14

K=3 S=2 P=1 → 14

K=5 S=2 P=2 → 14

K=7 S=2 P=3 → 14

네 조합 모두 출력이 14. 크기만으로는 못 고름

여기서 얻는 것

출력 크기는 stride 를 좁히는 데는 강하고, K 와 P 를 각각 정하는 데는 약함

K 와 P 는 짝으로만 결정됨 — 실제 코드에서는 층 정의를 봐야 확정할 수 있음

오늘 정리

stride	움기는 폭. 크면 출력이 작아짐
padding	바깥 채우기. 값은 관측값 아님
공식	$\text{floor}((I + 2P - D(K-1) - 1) / S + 1)$
floor	남는 칸은 버려짐
stride 2	계산량 약 1/4

지금까지의 전제

지금까지 다룬 입력 (1, 28, 28) 채널 1개
FashionMNIST 라서 흑백

컬러라면

필터는 어떻게 달라져야 하는가?

다채널 입력과 여러 필터

CNN · 챕터 8

앞 챕터까지 세운 것

창 하나	숫자 하나를 만듦
창 옮기기	stride
가장자리	padding
출력 크기	$\text{floor}((I + 2P - D(K-1) - 1) / S + 1)$

전부 **총이 1장**일 때의 이야기였음

오늘의 질문

컬러라면 필터는

어떻게 달라져야 하는가?

첫날 심어 둔 문장

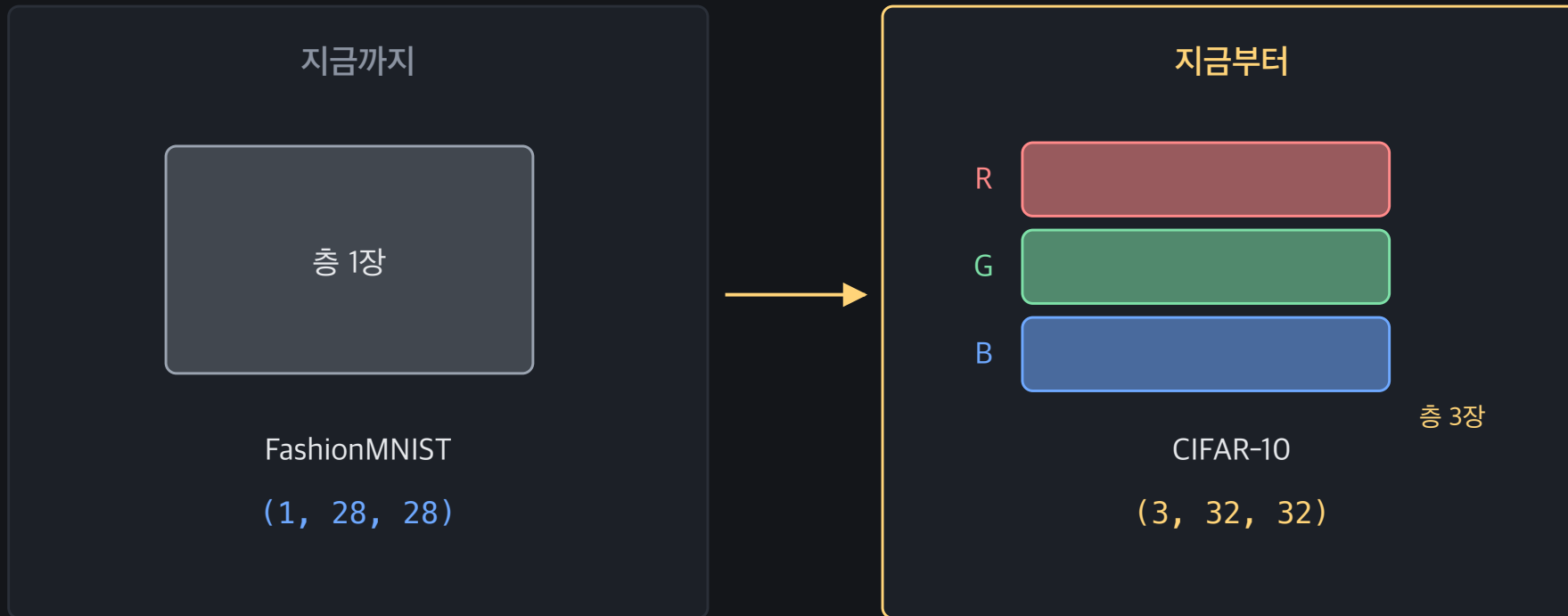
채널은 장수가 아니라 층수임

한 자리의 색을 알려면

세 층을 다 봐야 함

오늘 이 문장을 씀

지금까지 총은 1장이었다



블록 1에서 말했던 것 — 채널은 장수가 아니라 총수. 한 자리의 색을 알려면 세 총을 다 봐야 함
그러면 창 하나는 총 몇 개를 봐야 하는가

데이터가 바뀜

	FashionMNIST	CIFAR-10
한 장	(1, 28, 28)	(3, 32, 32)
숫자 개수	784	3,072
총	1장	3장
클래스	10	10

“ 총이 3장인 데이터가 필요해서 바꾼 것임

”

그러면

창 하나는

충 몇 개를 봐야 하는가?

먼저 각자 떠올려 보기

3×3 창을 3층짜리 입력 위에 놓는다면

- 커널은 몇 개 필요함?
- 출력은 몇 장 나옴?

대부분 같은 대답이 나옴

흔한 첫 대답

"RGB니까 커널도 3개 두고 각자 돌리면 되지 않나"



출력이 3장?

빨강 결과 · 초록 결과
파랑 결과

틀렸음

이렇게 하면 층끼리 한 번도 만나지 않음. 빨강 결과는 빨강만 보고 나온 값임
한 자리의 색을 알려면 세 층을 같이 봐야 하는데, 이 방식은 그걸 못 함
출력 map 하나가 입력 3채널 전부를 봐야 함

이 대답이 걸리는 지점

- 빨강 결과는 빨강 층만 보고 나온 값임
- 세 층이 **한 번도 만나지 않음**
- 그런데 색은 세 층이 만나야 생기는 것

출력 map 하나가 입력 3채널 전부를 봐야 함

실제로 일어나는 일

세 단계로 나눠서 봄

1단계	층마다 slice 한 장씩
2단계	셋을 더하고 bias 하나
3단계	이 묶음이 필터 1개

1단계 · 층마다 slice 한 장씩 않는다

입력 (3, 5, 5) · 창은 왼쪽 위 3×3 자리에 있다

R 층

2	0	1	3	1
3	1	0	2	0
1	2	0	1	2
0	3	1	0	1
2	1	2	1	0

이 층에 있는 slice

1	0	-1
1	0	-1
1	0	-1

→ 부분합
5

G 층

0	1	1	0	2
2	0	1	1	0
2	1	2	0	1
1	2	0	2	1
0	1	1	0	2

이 층에 있는 slice

1	1	1
0	0	0
-1	-1	-1

→ 부분합
-3

B 층

1	2	0	1	1
1	3	2	0	2
0	1	1	2	0
2	0	1	1	0
1	1	0	2	1

이 층에 있는 slice

0	1	0
1	0	1
0	1	0

→ 부분합
6

층 3장에 slice 3장. 여기까지는 따로 계산함

하지만 이 세 숫자가 각자 출력이 되는 것은 아님 — 다음 단계가 남았음

1단계에서 나온 것

숫자 3개 — 5, -3, 6

- 총마다 slice가 하나씩 없었음
- 여기까지는 총끼리 따로 계산함

“ 이 셋이 각자 출력이 되는 것은 아님

”

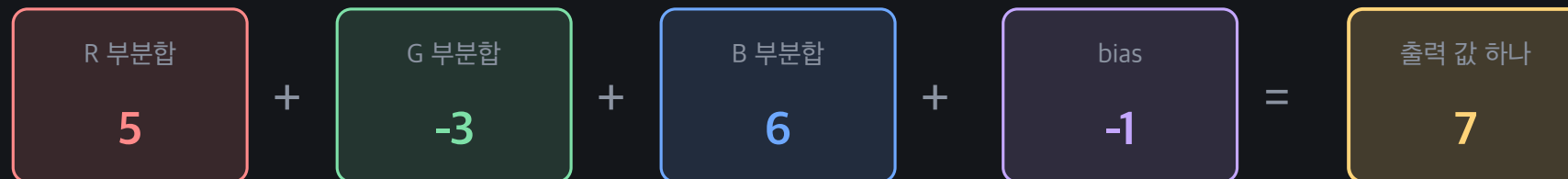
다음 질문

이 세 숫자를

어떻게 하는가?

2단계 · 셋을 더하고 bias 하나를 붙인다

출력 map의 값 한 개가 이렇게 만들어진다



창을 5×5 전체에 미끄러뜨리면

출력 map 한 장이 채워짐

7	-1	4
6	6	-2
7	6	4

(1, 3, 3)

입력 채널 3장 → 출력 채널 1장

출력 map 하나는 입력 3채널 전부를 봤음

bias는 slice마다가 아니라 이 출력 map 하나에 하나

2단계에서 일어난 일

$$5 + (-3) + 6 + (-1) = 7$$

- 총 3장의 결과가 **여기서 만남**
- bias는 slice마다가 아니라 **출력 map 하나에 하나**
- 입력 채널 3장 → 출력 채널 **1장**

출력 map 하나가

입력 3채널 전부를 봤음

첫날의 그 문장이 여기서 계산이 됨

그런데

출력이 한 장뿐이면

컬러를 흑백으로 만든 셈 아닌가?

3단계 · 이 묶음 전체가 필터 1개다

필터를 4개 두면 출력 map이 4장 나온다



필터 개수 = 출력 채널 수. 필터 하나가 출력 map 한 장을 만듦

필터 하나 안의 slice 개수 = 입력 채널 수. 이건 우리가 고르는 값이 아니라 입력이 정해 줌

3단계에서 정해지는 것

필터 하나 안의 slice 수	입력이 정함
필터 개수	내가 정함
출력 채널 수	필터 개수와 같음
bias 개수	출력 채널 수와 같음

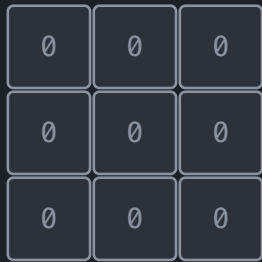
세 단계 한 줄씩

1단계	총 3장 × slice 3장 → 숫자 3개
2단계	3개를 더하고 bias 1개 → 출력 map 1장
3단계	이 묶음이 필터 1개. 필터 4개 → 출력 4장

3단계까지 가야 필터 하나가 완성됨

필터 하나는 납작한 사각형이 아니다

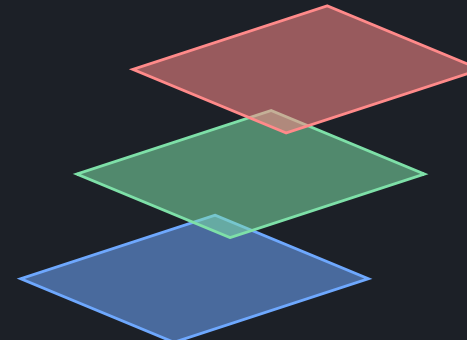
머릿속 그림 (틀림)



3×3 숫자 9개

= 필터 1개

실제 (맞음)



3×3 slice가 3장 겹친 묶음

숫자 27개 = 필터 1개

filter 하나는 입력 채널 수만큼 slice를 가짐. 입력이 3채널이면 slice도 3장
kernel_size=3 은 slice 한 장의 크기일 뿐 필터 전체의 크기가 아님

말이 헛갈리는 지점

- `kernel_size=3` 은 **slice 한 장**의 크기임
- 필터 하나의 크기가 아님
- 입력이 3채널이면 필터 하나에 숫자가 **27개**

“ 필터 · 커널이라는 말이 자료마다 다르게 쓰임. 축을 세어 확인하는 편이 안전함 ”

다음 질문

이걸 코드로는
어떻게 적는가?

한 줄임

```
conv = nn.Conv2d(in_channels=3, out_channels=4, kernel_size=3)
```

in_channels	입력 층이 몇 장 — 데이터가 정함
out_channels	필터 몇 개 — 내가 정함
kernel_size	slice 한 장의 크기

그래서 무게 텐서는 축이 4개다

```
weight.shape = (4, 3, 3, 3)
```

out_channels 4 필터 몇 개 — 내가 고름

in_channels 3 필터 하나의 slice 몇 장 — 입력이 정함

kH 3 slice 세로

kW 3 slice 가로

bias는 따로

bias.shape = (4,)

출력 채널마다 하나

slice마다가 아님

출력 map 한 장 전체에

같은 값 하나가 더해짐

groups=1 이 기본

모든 출력이 모든 입력 채널을 봄

축 네 개를 순서대로 읽기

(out, in, kH, kW)

```
conv.weight.shape    torch.Size([4, 3, 3, 3])  
conv.bias.shape      torch.Size([4])
```

앞의 두 자리가 **몇 장이 몇 장을 보는가**를 말해 줌

넣어 보면

```
x = torch.randn(2, 3, 32, 32)
conv = nn.Conv2d(3, 4, kernel_size=3, padding=1)
conv(x).shape
```

```
torch.Size([2, 4, 32, 32])
```

- **N** 은 그대로 2
- **C** 가 3에서 **4로 바뀜** — 필터 개수
- **H·W** 는 padding 덕분에 그대로

확인해 볼 것

정말 채널 3장을 더한 것과

같은 값이 나오는가?

손으로 나눠서 계산해 보기

```
part = 0
for c in range(3):
    part = part + F.conv2d(x[:, c:c+1], conv.weight[:, c:c+1],
                           bias=None, padding=1)
part = part + conv.bias.view(1, -1, 1, 1)

torch.allclose(conv(x), part, atol=1e-5)
```

True

이 결과가 뜻하는 것

- 채널 방향 합산이 `Conv2d` 안에서 일어남
- bias는 합친 뒤에 한 번 더해짐

“ 뜻하지 않는 것 — 실제 구현이 이 순서로 돈다는 말은 아님. 결과가 같다는 것까지임 ”

다음 질문

이 층은 값을 몇 개

배우는가?

손으로 세어 보기

```
Conv2d(3, 16, kernel_size=3)
```

필터 하나 = $3 \times 3 \times 3$ = ----

필터 16개 = ----

bias = ----

합계 = ----

파라미터 수는 곱셈 한 줄로 나온다

$$\text{파라미터} = \text{out} \times (\text{in} \times \text{kH} \times \text{kW}) + \text{out}$$

in	out	k	weight.shape	계산	합계
1	4	3	(4, 1, 3, 3)	$4 \times (1 \times 3 \times 3) + 4$	40
3	4	3	(4, 3, 3, 3)	$4 \times (3 \times 3 \times 3) + 4$	112
3	16	3	(16, 3, 3, 3)	$16 \times (3 \times 3 \times 3) + 16$	448
16	32	3	(32, 16, 3, 3)	$32 \times (16 \times 3 \times 3) + 32$	4,640
3	64	7	(64, 3, 7, 7)	$64 \times (3 \times 7 \times 7) + 64$	9,472

면적(H·W)은 파라미터 수에 들어가지 않음. 창은 자리를 옮겨 가며 같은 숫자를 다시 씀
in과 out을 둘 다 두 배로 하면 파라미터는 네 배가 됨

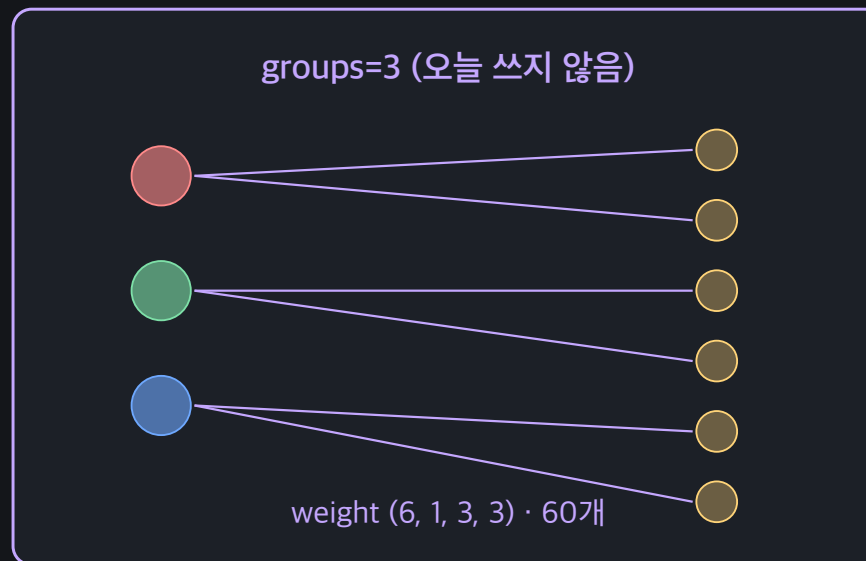
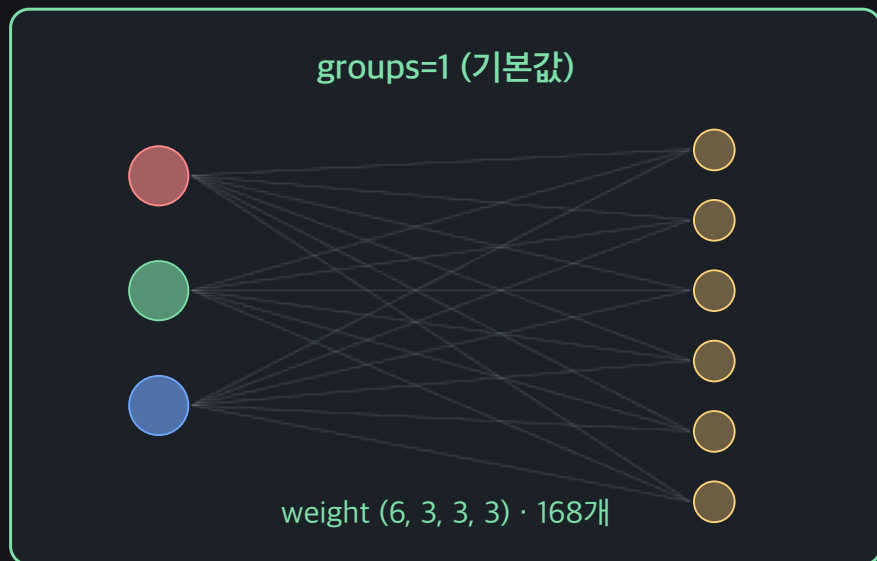
파라미터에 대해 확실한 것

- 공식은 $\text{out} \times (\text{in} \times k_H \times k_W) + \text{out}$
- **면적은 들어가지 않음** — 창이 자리를 옮겨 가며 같은 숫자를 다시 씀
- **in** 과 **out** 을 둘 다 두 배로 하면 파라미터는 **네 배**

“ 파라미터가 적다고 계산이 적은 것은 아님. 별개의 이야기임

”

groups=1 — 모든 출력이 모든 입력 채널을 본다



오늘 쓰는 모든 Conv2d는 groups=1. 출력 map 하나가 입력 채널 전부를 봄
groups를 나누면 층이 서로 안 만나게 됨 — 처음 떠올린 그 오답이 여기서도 이름 붙은 선택지가 됨

`groups=1` 이 기본값

- 출력 map 하나가 **모든 입력 채널**을 봄
- 오늘 쓰는 모든 `Conv2d` 가 이 설정임

처음 떠올렸던 "층마다 따로" 는 `groups` 를 나누면 실제로 가능함
기본값이 아니라는 것만 기억하기

오늘 정리 · 다음 질문

필터 하나	입력 채널 수만큼 slice
출력 map 하나	입력 채널 전부 를 봄
출력 채널 수	필터 개수
<code>weight.shape</code>	<code>(out, in, kH, kW)</code>

채널은 늘었는데 면적은 그대로임

이대로 쌓아도 되는가?